

Unit IV: Client-side scripting with JavaScript (10 Hrs.)

1. JavaScript Overview, Structure, Enabling, Placement, Variables, Data types, Operators, Control structure ,Type conversion, Functions, Arrays, Date and Math function, ES6 Fundamentals
 2. Element Access in JavaScript, Event handling, Error handling
 3. Form Validation with Regular Expression
 4. DOM and DOM tree
 5. Introduction to Asynchronous Programming (XMLHttpRequest)
-

Introduction:

- In the modern digital world, websites are expected to be interactive, responsive, and user-friendly.
- Users expect web pages to respond instantly to their actions, validate their inputs, display dynamic content, and communicate with servers without reloading the entire page. These capabilities are made possible through JavaScript.
- JavaScript is one of the most important technologies used in web development. Along with HTML and CSS, it forms the foundation of the World Wide Web.
 - HTML provides the structure of a webpage.
 - CSS provides the presentation and appearance.
 - JavaScript provides behavior and interactivity.
- Without JavaScript, websites would mostly consist of static pages that cannot respond dynamically to user actions.
- It was developed in 1995 by Brendan Eich for the Netscape browser and later standardized as ECMAScript.
- Today, JavaScript is widely used not only in web development but also in server-side programming, mobile applications, and desktop applications.

Definition

JavaScript is a high-level, interpreted, object-based scripting language used to create dynamic and interactive web pages by enabling client-side execution within web browsers.

It allows web developers to:

- Add interactivity to web pages
- Manipulate HTML and CSS content dynamically
- Validate user input in forms
- Handle events such as clicks and keypresses
- Communicate with servers without reloading the page

In simple terms, JavaScript is the programming language that makes a static webpage behave like a dynamic and interactive application.

Features of JavaScript

1. **Lightweight Language:** JavaScript is simple and requires less memory and system resources.
2. **Interpreted Language:** Code is executed line by line directly by the browser without compilation.
3. **Object-Based / Object-Oriented Support:** Supports objects, properties, and methods for structured programming.
4. **Event-Driven Language:** Responds to user actions such as clicks, key presses, and mouse movements.
5. **Dynamic Typing:** Variables can hold different types of data during execution.
6. **Platform Independent:** Runs on any operating system with a compatible web browser.
7. **Client-Side Execution:** Mostly executed in the browser, reducing server load.
8. **Case Sensitive:** JavaScript treats uppercase and lowercase as different identifiers.
9. **Prototype-Based Inheritance:** Objects can inherit properties and behavior through prototypes.
10. **Asynchronous Processing Support:** Supports non-blocking operations like timers and server requests.

History of JavaScript

Year	Event / Version	Description
1995	JavaScript Invented	Created by Brendan Eich at Netscape in about 10 days to add interactivity to web pages. Initially named Mocha, then LiveScript, and finally JavaScript.
1996	JavaScript 1.0	First version released in Netscape Navigator 2.0. Supported basic scripting and form validation.
1997	Standardization (ECMA-262)	JavaScript submitted to ECMA (European Computer Manufacturers Association) International and standardized as ECMAScript.
1997	ECMAScript 1 (ES1)	First official standard version of JavaScript.
1998	ECMAScript 2 (ES2)	Minor updates and editorial improvements.
1999	ECMAScript 3 (ES3)	Major improvements including regular expressions, better error handling, and enhanced strings.
2008	ECMAScript 4 (ES4)	Proposed but later abandoned due to complexity and disagreements.
2009	ECMAScript 5 (ES5)	Introduced strict mode, JSON support, and new array methods. Became widely adopted.
2015	ECMAScript 6 (ES6)	Major update introducing modern features like let, const, arrow functions, classes, modules, and template literals.
Present	Modern JavaScript	Continuous yearly updates; widely used in web, mobile, desktop, and server-side development.

Why JavaScript is called a Client-Side Scripting Language?

- JavaScript is called a **client-side scripting language** because it is executed on the **user's web browser (client machine)** instead of being executed on the web server.
- When a user requests a webpage, the server sends HTML, CSS, and JavaScript code to the browser.
- The browser then processes and executes the JavaScript code locally on the user's system to make the webpage interactive and dynamic.
- Since the execution of JavaScript happens on the client side, it is known as a client-side scripting language.

Reasons:

1. **Execution in Browser:** JavaScript runs directly in web browsers such as Chrome, Firefox, and Edge.

2. **No Server Processing Required:** Tasks like form validation and simple calculations are performed in the browser.
3. **Fast Response Time:** Since execution is local, results are displayed quickly without waiting for the server.
4. **Reduces Server Load:** Server workload is reduced because simple operations are handled by the client.
5. **Improves User Interaction:** It enables dynamic and interactive web pages such as buttons, forms, and menus.

Example:

```
<button onclick="alert('Hello')">Click Me</button>
```

When the button is clicked, the JavaScript code runs in the browser without communicating with the server.

Thus, JavaScript is called a client-side scripting language because it is executed in the user's browser, enabling interactive and dynamic web pages efficiently.

Applications of JavaScript

1. Web development (interactive websites)
2. Form validation
3. Web applications (e.g., Gmail, Facebook)
4. Game development (browser-based games)
5. Mobile application development (React Native)
6. Server-side development (Node.js)
7. Desktop applications (Electron apps)

Advantages of JavaScript

1. Easy to learn
2. Fast execution in browser
3. Reduces server load
4. Platform independent
5. Creates interactive web pages
6. Large community support and libraries

Limitations of JavaScript

1. Security issues (code visible in browser)
2. Browser dependency
3. Limited access to system resources
4. Different behavior in different browsers
5. Single-threaded execution
6. Client-side code can be disabled by user

Enabling JavaScript

- JavaScript must be enabled in a web browser in order to execute scripts and provide interactive features on web pages.
- By default, JavaScript is enabled in most modern browsers, but it can be disabled by users for security or privacy reasons.

Methods of Enabling JavaScript

1. In Google Chrome

- Open **Settings**
- Go to **Privacy and Security**
- Select **Site Settings**
- Click on **JavaScript**
- Select **Sites can use JavaScript**

2. In Mozilla Firefox

- Type about:config in the address bar
- Search for javascript.enabled
- Set value to **true**

3. In Microsoft Edge

- Open **Settings**
- Go to **Cookies and Site Permissions**
- Click on **JavaScript**
- Enable **Allowed (recommended)** option

4. In Safari (Mac)

- Open **Preferences**
- Go to **Security tab**
- Check **Enable JavaScript**

Importance of Enabling JavaScript

1. Allows dynamic and interactive web pages
2. Enables form validation and event handling
3. Supports modern web applications
4. Required for proper functioning of most websites

Placement of JavaScript (Ways to implements JS in HTML)

JavaScript can be placed or implemented in an HTML document in different ways. The placement determines how and when the JavaScript code is executed in the web page.

1. Inline JavaScript

- Inline JavaScript is a method of implementing JavaScript in which the code is written **directly inside HTML elements** using event attributes such as onclick, onmouseover, onchange, etc.
- It is mainly used to perform **small and simple tasks**.
- **Syntax:** `<element event=" JavaScript code">`

Example

```
<!DOCTYPE html>
<html>
<body>

<h2>Inline JavaScript Example</h2>

<button onclick="alert('Welcome to JavaScript')">
  Click Me
</button>

</body>
</html>
```

Common Event Attributes Used

Event	Description
onclick	Executes when an element is clicked
ondblclick	Executes on double click
onmouseover	Executes when mouse is over an element
onchange	Executes when input value changes
onload	Executes when page loads

Advantages

1. Very simple to use
2. Quick implementation
3. Useful for small tasks and testing

Disadvantages

1. Poor code organization
2. Not reusable
3. Difficult to maintain in large applications
4. Mixes HTML and JavaScript together

2. Internal JavaScript

Internal JavaScript is a method of implementing JavaScript in which the JavaScript code is written inside the same HTML document using the <script> tag.

The <script> tag can be placed either in the <head> section or the <body> section of the HTML document.

Syntax

```
<script>
  JavaScript Code
</script>
```

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JavaScript </title>
</head>
<body>

  <h2>Internal JavaScript Example</h2>

  <button onclick="showMessage()">
    Click Me
  </button>

  <script>
    function showMessage() {
      alert("Welcome to JavaScript");
    }
  </script>

</body>
</html>
```

Placement of Internal JavaScript

1. Inside the <head> Section

```
<head>
  <script>
    function greet() {
      alert("Hello Students");
    }
  </script>
</head>
```

2. Inside the <body> Section

```
<body>

  <script>
    document.write("Welcome");
```

</script>

</body>

The <body> placement is generally preferred because the HTML elements are loaded before the script executes.

Characteristics of Internal JavaScript

- Code is written within the HTML file.
- Uses the <script> tag.
- Suitable for small and medium-sized web pages.
- Easy to test and modify.

Advantages

1. Easy to implement.
2. No separate JavaScript file is needed.
3. Suitable for small web applications.
4. Faster access because code is stored in the same file.

Disadvantages

1. Code cannot be reused across multiple pages.
2. Makes HTML files larger.
3. Difficult to maintain in large projects.
4. Mixing HTML and JavaScript reduces readability.

3. External JavaScript

External JavaScript is a method of implementing JavaScript in which the JavaScript code is written in a **separate file** with a **.js extension** and linked to an HTML document using the <script> tag and the src attribute.

This is the most commonly used and recommended method for developing web applications because it improves code organization, reusability, and maintainability.

Syntax

HTML File

```
<script src="filename.js"></script>
```

JavaScript File

```
// JavaScript code
```

Example

HTML File (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>External JavaScript Example</title>
</head>
<body>

  <h2>External JavaScript Example</h2>

  <button onclick="showMessage()">
    Click Me
  </button>

  <script src="script.js"></script>

</body>
</html>
```

JavaScript File (script.js)

```
function showMessage() {
  alert("Welcome to JavaScript");
}
```

Characteristics of External JavaScript

- JavaScript code is stored in a separate .js file.
- Linked to HTML using the src attribute.
- One JavaScript file can be used by multiple webpages.
- Separates content (HTML) from behavior (JavaScript).

Advantages

1. **Code Reusability:** The same JavaScript file can be used in multiple webpages.
2. **Easy Maintenance:** Changes need to be made in only one file.
3. **Better Code Organization:** Keeps HTML and JavaScript separate.
4. **Improved Readability:** HTML files become cleaner and easier to understand.
5. **Faster Loading:** Browsers can cache external JavaScript files, reducing loading time on subsequent visits.
6. **Suitable for Large Projects:** Makes large applications easier to manage.

Disadvantages

1. Requires an additional file.
2. If the JavaScript file is missing, the script will not work.
3. An extra HTTP request is needed to load the file.
4. Internet connection may be required if the file is hosted externally.

Comparison of Inline, Internal, and External JavaScript

Inline JavaScript	Internal JavaScript	External JavaScript
JavaScript code is written directly inside an HTML element.	JavaScript code is written inside the <script> tag in the HTML file.	JavaScript code is written in a separate .js file.
Located within HTML tags.	Located inside the HTML document.	Located in a separate JavaScript file.
Cannot be reused in other webpages.	Has limited reusability.	Can be reused in multiple webpages.
Difficult to maintain.	Easier to maintain than inline JavaScript.	Very easy to maintain.
Poor code organization.	Better code organization.	Best code organization.
Increases the size of the HTML file.	Increases the size of the HTML file.	Keeps the HTML file smaller.
Suitable for small tasks and simple actions.	Suitable for small web applications.	Suitable for medium and large web applications.
Less readable.	Moderately readable.	Highly readable.
Not recommended for large projects.	Suitable for small projects.	Most recommended method for web development.

JavaScript Variables

- A **variable** in JavaScript is a named container used to store data values.
- These values can be changed, reused, and manipulated during program execution.
- Variables form the basic foundation of programming logic and are essential for storing user input, performing calculations, and controlling program flow.
- A variable is a memory location that holds a value which can be modified during program execution (depending on the declaration type).

Syntax : let variableName = value;

Example

```
let name = "Ram";
```

```
let age = 20;
```

Rules for Variable Naming

- Variable names must begin with a letter (A–Z or a–z), underscore (_), or dollar sign (\$)
- Variable names cannot start with a number
- Variable names cannot contain spaces
- Variable names are case-sensitive
- Reserved keywords (e.g., let, var, function, if) cannot be used as variable names
- Variable names should not contain special characters except _ and \$
- Variable names should be meaningful and descriptive
- CamelCase convention is recommended for naming variables (e.g., studentName, totalMarks)
- Variable names should not be too short or unclear (avoid single-letter names in real programs)
- Duplicate variable names are not allowed in the same scope when using let or const

Types of Variables in JavaScript

JavaScript provides three main ways to declare variables. These determine how data is stored, accessed, and updated in a program.

1. `var`
2. `let`
3. `const`

1. **var:** `var` is the traditional keyword used to declare variables in JavaScript. It was used before ES6 and is still supported for backward compatibility.

Characteristics

- Function-scoped
- Can be redeclared in the same scope
- Can be reassigned
- Hoisted (initialized as undefined)
- Less strict and more error-prone compared to modern alternatives

Syntax: `var variableName = value;`

Example

```
var city = "Kathmandu"; // initial declaration
var city = "Pokhara";   // redeclaration allowed
city = "Lalitpur";      // reassignment allowed
```

```
console.log(city);
```

Output: Lalitpur

2. **let:** `let` is an ES6 feature used to declare variables with block scope, providing better control and reducing programming errors.

Characteristics

- Block-scoped
- Cannot be redeclared in the same scope
- Can be reassigned
- Avoids common issues found in `var`
- Hoisted but not initialized (Temporal Dead Zone)

Syntax: `let variableName = value;`

Example

```
let age = 20;    // declaration
age = 25;       // reassignment allowed
```

```
{
  let age = 30; // different block scope
  console.log(age);
}
console.log(age);
```

Output

```
30
25
```

3. **const:** const is used to declare constant values in JavaScript. Once assigned, the value cannot be changed.

Characteristics

- Block-scoped
- Cannot be redeclared
- Cannot be reassigned
- Must be initialized at the time of declaration
- Used for fixed values

Syntax : `const variableName = value;`

Example

```
const country = "Nepal"; // initialization required
console.log(country);
// country = "India";    // Not allowed (error)
// const country = "USA"; // Not allowed (error)
```

Output: Nepal

Comparison of var, let, and const

var	let	const
Function-scoped variable declaration.	Block-scoped variable declaration.	Block-scoped constant declaration.
Allows redeclaration within the same scope.	Redeclaration within the same scope is not permitted.	Redeclaration is not permitted.
Supports reassignment of values.	Supports reassignment of values.	Does not support reassignment after initialization.
Initialization is optional at the time of declaration.	Initialization is optional at the time of declaration.	Initialization is mandatory at the time of declaration.
Considered outdated and generally discouraged in modern development.	Recommended for variables whose values are expected to change.	Recommended for values that remain constant throughout program execution.

Data Types in JavaScript

- Data types in JavaScript define the type of value that a variable can store and determine the operations that can be performed on it.
- JavaScript is a **dynamically typed language**, meaning that a variable can hold different types of values during execution without explicit type declaration.
- A data type is a classification of data that specifies what type of value a variable can store and how that value behaves in operations.
- JavaScript data types are mainly classified into:
 1. Primitive Data Types
 2. Non-Primitive (Reference) Data Types

Primitive Data Types

- Primitive data types are the most fundamental data types in JavaScript.
- They are used to store single values and form the basic building blocks for data handling in programs.
- JavaScript is a dynamically typed language, so variables can store different primitive types without explicit type declaration.
- Primitive data types are immutable, meaning their values cannot be modified once created. Any modification results in the creation of a new value in memory.
- JavaScript supports seven primitive data types.

Primitive Data Types

Name	Description	Example
Number	Represents both integer and floating-point numeric values	let age = 20;
String	Represents a sequence of characters enclosed in quotes	let name = "Ram";
Boolean	Represents logical values: true or false	let isStudent = true;
Undefined	Represents a variable that is declared but not assigned a value	let x;
Null	Represents intentional absence of any value	let data = null;
Symbol	Represents a unique and immutable identifier introduced in ES6	let id = Symbol("unique");
BigInt	Represents integers larger than the safe limit of Number type	let bigNumber = 123456789012345678901234567890n;

Example

```
let age = 20;
let name = "Sita";
let isStudent = true;
let value;
let data = null;
let id = Symbol("id");
let bigNumber = 123456789012345678901234567890n;
console.log(typeof age);
console.log(typeof name);
console.log(typeof isStudent);
console.log(typeof value);
console.log(typeof data);
console.log(typeof id);
```

```
console.log(typeof bigNumber);
```

Non-Primitive (Reference) Data Types

- Non-primitive data types in JavaScript are used to store **complex data and multiple values**.
- Unlike primitive data types, they do not store the actual value directly.
- Instead, they store a **reference (memory address)** pointing to the location where the data is stored.
- These data types are **mutable**, meaning their values can be modified after creation.
- Non-primitive data types are data types that store collections of values or complex structures. They are accessed through references rather than storing the actual value directly.
- Non-primitive data types in JavaScript mainly include:
 1. Object
 2. Array
 3. Function

Characteristics

- Stored by reference (memory address is stored, not actual value)
- Mutable in nature (values can be changed)
- Can store multiple values or complex structures
- Dynamic size (can grow or shrink during execution)
- Comparatively slower access than primitive data types
- Multiple variables can refer to the same memory location

- 1) **Object:** An object is a collection of related data stored in the form of **key-value pairs**. It is used to represent real-world entities.

Example

```
let student = {  
  name: "Sita",  
  age: 21,  
  faculty: "IT"  
};
```

```
console.log(student.name);
```

Output: Sita

- 2) **Array:** An array is a special type of object used to store **multiple values in a single variable**. Values are stored in indexed positions starting from 0.

Example

```
let fruits = ["Apple", "Banana", "Mango"];  
console.log(fruits[0]);
```

Output: Apple

- 3) **Function:** A function is a reusable block of code designed to perform a specific task. In JavaScript, functions are also treated as objects.

Example

```
function add(a, b) {
```

```
    return a + b;  
}  
console.log(add(5, 3));
```

Output: 8

Reference Behavior

- Reference behavior in JavaScript explains how **non-primitive data types (objects, arrays, and functions)** are stored and accessed in memory.
- Unlike primitive data types, which store actual values, non-primitive data types store a **reference (memory address)**.
- Reference behavior refers to the mechanism in JavaScript where variables assigned to non-primitive data types store and share the **memory address of the data**, rather than copying the actual value.
- When a non-primitive value is assigned to a variable:
 - The actual data is stored in memory (heap memory)
 - The variable stores the **reference (address)** of that memory
 - If another variable is assigned the same reference, both point to the same data

Example :

```
let obj1 = { name: "Ram" };  
let obj2 = obj1;  
obj2.name = "Sita";  
console.log(obj1.name);  
Output: Sita
```

Both variables refer to the same memory location. Therefore, changing the value through one variable affects the other.

Operators in JavaScript

- Operators in JavaScript are special symbols used to perform operations on variables and values.
- They are used to construct expressions, perform computations, compare values, and control program logic.
- Operators are essential in programming as they enable data manipulation and decision-making.

Operand: The value or variable on which operation is performed

Operator: The symbol that performs the operation

Example: `let result = 10 + 5;`

Here, `+` is the operator and `10`, `5` are operands.

Types of Operators

Arithmetic Operators: Arithmetic operators are used to perform mathematical operations.

Operator	Meaning	Example
<code>+</code>	Addition	<code>10 + 5 = 15</code>
<code>-</code>	Subtraction	<code>10 - 5 = 5</code>
<code>*</code>	Multiplication	<code>10 * 5 = 50</code>
<code>/</code>	Division	<code>10 / 5 = 2</code>
<code>%</code>	Modulus (remainder)	<code>10 % 3 = 1</code>
<code>**</code>	Exponentiation	<code>2 ** 3 = 8</code>

Example

```
let a = 10;
let b = 5;
console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
```

Assignment Operators: Assignment operators are used to assign values to variables.

Operator	Meaning
<code>=</code>	Assign
<code>+=</code>	Add and assign
<code>-=</code>	Subtract and assign
<code>*=</code>	Multiply and assign
<code>/=</code>	Divide and assign

Example

```
let x = 10;
```

```
x += 5; // x = x + 5
console.log(x);
```

Comparison Operators: Comparison operators are used to compare two values and return a Boolean result (true or false).

Operator	Meaning	Example
==	Equal to	5 == "5" → true
===	Strict equal (value + type)	5 === "5" → false
!=	Not equal	5 != 3 → true
>	Greater than	10 > 5 → true
<	Less than	5 < 10 → true
>=	Greater than or equal	10 >= 10 → true
<=	Less than or equal	5 <= 10 → true

Example

```
console.log(10 == "10");
console.log(10 === "10");
console.log(10 > 5);
```

Logical Operators: Logical operators are used to combine multiple conditions.

Operator	Name	Description
&&	Logical AND	Returns true if all conditions are true
	Logical OR	Returns true if at least one condition is true
!	Logical NOT	Reverses the Boolean value

Example

```
let a = 10;
let b = 5;
console.log(a > 5 && b < 10);
console.log(a > 5 || b > 10);
console.log(!(a > 5));
```

Increment and Decrement Operators: These operators are used to increase or decrease a value by 1.

Operator	Name	Description
++	Increment	Increases value by 1
--	Decrement	Decreases value by 1

Example

```
let x = 5;
x++;
console.log(x);
x--;
console.log(x);
```

Conditional (Ternary) Operator

The conditional (ternary) operator is a JavaScript operator that evaluates a condition and returns one value if the condition is true and another value if the condition is false.

Syntax

```
condition ? expression1 : expression2;
```

Example

```
let age = 18;
```

```
let result = (age >= 18) ? "Eligible to vote" : "Not eligible";
```

```
console.log(result);
```


typeof Operator

The typeof operator is a built-in JavaScript operator used to determine the data type of a variable or value. It is commonly used for validation, debugging, and runtime type checking.

Syntax

typeof variableName
or

typeof(value)

Example

```
let name = "Ram";  
let age = 20;  
  
console.log(typeof name);  
console.log(typeof age);
```

Output

```
string  
number
```

Class work

- 1) Write a JavaScript program to display “Hello World”.

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Hello World Program</title>  
</head>  
<body>  
  
  <script>  
    document.write("Hello World");  
  </script>  
  
</body>  
</html>
```

- 2) Write a JavaScript program to display student details (name, age, address).

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Student Details</title>  
</head>  
<body>  
  
  <script>  
    let name = "Ram Sharma";  
    let age = 20;
```

```

let address = "Kathmandu";

document.write("Student Details <br>");
document.write("Name: " + name + "<br>");
document.write("Age: " + age + "<br>");
document.write("Address: " + address + "<br>");
</script>

</body>
</html>

```

3) Write a JavaScript program to demonstrate variable declaration using var, let, and const.

```

<!DOCTYPE html>
<html>
<head>
  <title>Variable Declaration</title>
</head>
<body>

<script>
  var a = 10;
  let b = 20;
  const c = 30;

  document.write("Demonstration of Variables <br><br>");

  document.write("Value of var a = " + a + "<br>");
  document.write("Value of let b = " + b + "<br>");
  document.write("Value of const c = " + c + "<br>");
</script>

</body>
</html>

```

4) Write a JavaScript program to display the data type of a variable using typeof operator.

```

<!DOCTYPE html>
<html>
<head>
  <title>Data Type using typeof</title>
</head>
<body>

<script>
  let a = "Hello";
  let b = 100;
  let c = true;
  let d;

```

```

document.write("Data Types of Variables <br><br>");

document.write("a = " + a + " → " + typeof a + "<br>");
document.write("b = " + b + " → " + typeof b + "<br>");
document.write("c = " + c + " → " + typeof c + "<br>");
document.write("d = " + d + " → " + typeof d + "<br>");
</script>

</body>
</html>

```

5) Write a JavaScript program to perform basic arithmetic operations.

```

<!DOCTYPE html>
<html>
<head>
  <title>Arithmetic Operations</title>
</head>
<body>
<script>
  let a = 20;
  let b = 10;
  let sum = a + b;
  let diff = a - b;
  let mul = a * b;
  let div = a / b;
  let mod = a % b;
  document.write("Arithmetic Operations <br><br>");
  document.write("a = " + a + ", b = " + b + "<br><br>");
  document.write("Addition: " + sum + "<br>");
  document.write("Subtraction: " + diff + "<br>");
  document.write("Multiplication: " + mul + "<br>");
  document.write("Division: " + div + "<br>");
  document.write("Modulus: " + mod + "<br>");
</script>

</body>
</html>

```

Branching Statements in JavaScript

- Branching statements are control structures that allow a program to make decisions and execute different blocks of code based on specified conditions.
- They help the program choose one path from multiple possible paths depending on the result of a condition.
- A branching statement is a decision-making statement that controls the flow of program execution by selecting a specific block of code based on a condition.

Need for Branching Statements

- To make decisions in a program.
- To execute different actions under different conditions.

- To validate user input.
- To implement logical operations in applications.
- To control program flow effectively.

Types of Branching Statements in JavaScript

1. if Statement
2. if...else Statement
3. if...else if...else Statement
4. Nested if Statement
5. switch Statement

1. if Statement

The if statement executes a block of code only when the specified condition is true.

Syntax

```
if (condition) {  
    // statements  
}
```

Example

```
let age = 20;
```

```
if (age >= 18) {  
    document.write("Eligible to vote");  
}
```

Advantages

- Simple and easy to use.
- Suitable for single-condition decisions.

Limitation

- Cannot handle alternative actions when the condition is false.

2. if...else Statement

The if...else statement provides two alternative execution paths. One block executes if the condition is true, and another block executes if the condition is false.

Syntax

```
if (condition) {  
    // statements if true  
}  
else {  
    // statements if false  
}
```

Example

```
let number = 7;

if (number % 2 == 0) {
    document.write("Even Number");
}
else {
    document.write("Odd Number");
}
```

Advantages

1. Provides two-way decision making.
2. Easy to understand and implement.

Limitation

1. Suitable only when there are two possible outcomes.

3. if...else if...else Statement

The if...else if...else statement is used when multiple conditions need to be tested.

Syntax

```
if (condition1) {
    // statements
}
else if (condition2) {
    // statements
}
else if (condition3) {
    // statements
}
else {
    // statements
}
```

Example

```
let marks = 75;

if (marks >= 80) {
    document.write("Distinction");
}
else if (marks >= 60) {
    document.write("First Division");
}
else if (marks >= 40) {
    document.write("Pass");
}
else {
    document.write("Fail");
}
```

```
}
```

Advantages

1. Handles multiple conditions.
2. Improves decision-making capability.

Limitation

1. Becomes difficult to manage when many conditions exist.

4. Nested if Statement

A Nested if statement is an if statement placed inside another if statement.

Syntax

```
if (condition1) {  
  
    if (condition2) {  
        // statements  
    }  
  
}
```

```
}
```

Example

```
let age = 20;  
let citizen = true;  
  
if (age >= 18) {  
  
    if (citizen == true) {  
        document.write("Eligible to Vote");  
    }  
  
}
```

```
}
```

Advantages

1. Useful for checking multiple related conditions.
2. Provides detailed decision-making.

Limitation

1. Can become complex and difficult to read.

5. switch Statement

The switch statement selects one block of code from multiple alternatives based on the value of an expression.

Syntax

```
switch(expression) {  
  
    case value1:
```

```
    statements;
    break;

case value2:
    statements;
    break;

default:
    statements;
}
```

Example

```
let day = 3;

switch(day) {

    case 1:
        document.write("Sunday");
        break;

    case 2:
        document.write("Monday");
        break;

    case 3:
        document.write("Tuesday");
        break;

    default:
        document.write("Invalid Day");
}
```

Advantages

1. Suitable for multiple fixed choices.
2. Improves readability compared to long if...else if chains.

Limitation

1. Works best with exact values.
2. Not suitable for complex conditions.

Looping Statements in JavaScript

- Looping statements are control structures that repeatedly execute a block of code until a specified condition becomes false. They help reduce code duplication, save development time, and improve program efficiency.
- Looping statements are widely used in JavaScript for performing repetitive tasks such as displaying numbers, processing arrays, and handling data collections.
- A looping statement is a programming construct that repeatedly executes a block of code as long as a given condition is true.

Types of Looping Statements in JavaScript

1. for Loop
2. while Loop
3. do...while Loop

1. for Loop

The for loop is used when the number of iterations is known in advance. It combines initialization, condition checking, and updating in a single statement.

Syntax

```
for(initialization; condition; increment/decrement)
{
    // statements
}
```

Example

```
for(let i = 1; i <= 5; i++)
{
    document.write(i + "<br>");
}
```

Output

```
1
2
3
4
5
```

Features

- Entry-controlled loop.
- Suitable when the number of iterations is known.
- Initialization, condition, and update are written together.

Advantages

1. Easy to understand and maintain.
2. Reduces code repetition.
3. Suitable for counting operations.

Limitations

1. Not suitable when the number of iterations is unknown.
2. Can become complex with multiple conditions.

2. while Loop

The while loop repeatedly executes a block of code as long as the specified condition remains true.

Syntax

```
while(condition)
{
    // statements
}
```

Example

```
let i = 1;
```

```
while(i <= 5)
{
    document.write(i + "<br>");
    i++;
}
```

Output

```
1
2
3
4
5
```

Features

- Entry-controlled loop.
- Condition is checked before execution.
- Useful when the number of iterations is unknown.

Advantages

1. Simple and flexible.
2. Suitable for condition-based repetition.
3. Easy to implement.

Limitations

1. May result in an infinite loop if the condition is never false.

2. Loop variable must be updated manually.

3. do...while Loop

The do...while loop executes a block of code first and then checks the condition. Therefore, the loop body executes at least once regardless of the condition.

Syntax

```
do
{
    // statements
}
while(condition);
```

Example

```
let i = 1;
```

```
do
{
    document.write(i + "<br>");
    i++;
}
while(i <= 5);
```

Output

```
1
2
3
4
5
```

Features

1. Exit-controlled loop.
2. Executes at least once.
3. Condition is checked after execution.

Advantages

1. Guarantees at least one execution.
2. Useful for menu-driven applications and input validation.

Limitations

1. May execute unnecessarily when the condition is false.
2. Less commonly used than for and while loops.

Write a JavaScript program to check whether a number is even or odd.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));

if(num % 2 == 0)
{
    document.write("Even Number");
}
else
{
    document.write("Odd Number");
}
</script>

</body>
</html>
```

Write a JavaScript program to find the largest of two numbers.

```
<!DOCTYPE html>
<html>
<body>

<script>
let a = parseInt(prompt("Enter first number:"));
let b = parseInt(prompt("Enter second number:"));

if(a > b)
{
    document.write(a + " is largest");
}
else
{
    document.write(b + " is largest");
}
</script>

</body>
</html>
```

Write a JavaScript program to find the largest of three numbers.

```
<!DOCTYPE html>
<html>
<body>

<script>
let a = parseInt(prompt("Enter first number:"));
let b = parseInt(prompt("Enter second number:"));
let c = parseInt(prompt("Enter third number:"));

if(a > b && a > c)
{
    document.write(a + " is largest");
}
else if(b > c)
{
    document.write(b + " is largest");
}
else
{
    document.write(c + " is largest");
}
</script>

</body>
</html>
```

Write a JavaScript program to check whether a number is positive, negative, or zero.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));

if(num > 0)
{
    document.write("Positive Number");
}
else if(num < 0)
{
    document.write("Negative Number");
}
else
{
    document.write("Zero");
}
</script>

</body>
```

</html>

Write a JavaScript program to display weekday using switch statement.

<!DOCTYPE html>

<html>

<body>

<script>

let day = parseInt(prompt("Enter day number (1-7):"));

switch(day)

{

case 1:

document.write("Sunday");

break;

case 2:

document.write("Monday");

break;

case 3:

document.write("Tuesday");

break;

case 4:

document.write("Wednesday");

break;

case 5:

document.write("Thursday");

break;

case 6:

document.write("Friday");

break;

case 7:

document.write("Saturday");

break;

default:

document.write("Invalid Day");

}

</script>

</body>

</html>

Write a JavaScript program to print numbers from 1 to 10 using for loop.

```
<!DOCTYPE html>
<html>
<body>

<script>
for(let i = 1; i <= 10; i++)
{
    document.write(i + "<br>");
}
</script>

</body>
</html>
```

Write a JavaScript program to print even numbers from 1 to 20.

```
<!DOCTYPE html>
<html>
<body>

<script>
for(let i = 2; i <= 20; i += 2)
{
    document.write(i + "<br>");
}
</script>

</body>
</html>
```

Write a JavaScript program to print odd numbers from 1 to 20.

```
<!DOCTYPE html>
<html>
<body>

<script>
for(let i = 1; i <= 20; i += 2)
{
    document.write(i + "<br>");
}
</script>

</body>
</html>
```

Write a JavaScript program to find the sum of first 10 natural numbers.

```
<!DOCTYPE html>
<html>
<body>

<script>
let sum = 0;

for(let i = 1; i <= 10; i++)
{
    sum = sum + i;
}

document.write("Sum = " + sum);
</script>

</body>
</html>
```

Write a JavaScript program to display multiplication table of a number.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));

for(let i = 1; i <= 10; i++)
{
    document.write(num + " x " + i + " = " + (num * i) + "<br>");
}
</script>

</body>
</html>
```

Write a JavaScript program to find factorial of a number.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));
let fact = 1;

for(let i = 1; i <= num; i++)
{
    fact = fact * i;
}

document.write("Factorial = " + fact);
</script>
```

```
</body>
</html>
```

Write a JavaScript program to reverse a number.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));
let reverse = 0;

while(num > 0)
{
    let digit = num % 10;
    reverse = reverse * 10 + digit;
    num = parseInt(num / 10);
}

document.write("Reverse = " + reverse);
</script>

</body>
</html>
```

Write a JavaScript program to check whether a number is prime or not.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));
let isPrime = true;

if(num <= 1)
{
    isPrime = false;
}

for(let i = 2; i < num; i++)
{
    if(num % i == 0)
    {
        isPrime = false;
        break;
    }
}

if(isPrime)
{
```



```
    document.write("Prime Number");
}
else
{
    document.write("Not Prime Number");
}
</script>
```

```
</body>
</html>
```

Write a JavaScript program to check whether a number is palindrome or not.

```
<!DOCTYPE html>
<html>
<body>

<script>
let num = parseInt(prompt("Enter a number:"));
let temp = num;
let reverse = 0;

while(num > 0)
{
    let digit = num % 10;
    reverse = reverse * 10 + digit;
    num = parseInt(num / 10);
}

if(temp == reverse)
{
    document.write("Palindrome Number");
}
else
{
    document.write("Not Palindrome Number");
}
</script>

</body>
</html>
```

Write a JavaScript program to display Fibonacci series.

```
<!DOCTYPE html>
<html>
<body>

<script>
let n = parseInt(prompt("Enter number of terms:"));

let a = 0, b = 1;

document.write(a + "<br>");
document.write(b + "<br>");

for(let i = 3; i <= n; i++)
{
    let c = a + b;
    document.write(c + "<br>");

    a = b;
    b = c;
}
</script>

</body>
</html>
```

Type Conversion in JavaScript

Type conversion is the process of transforming a value from one data type into another. Since JavaScript is a dynamically typed language, data often needs to be converted to a compatible type before performing operations. Type conversion helps ensure that data is processed correctly and efficiently within a program.

Types of Type Conversion

JavaScript supports two types of type conversion:

- 1) Implicit Type Conversion
- 2) Explicit Type Conversion

Implicit Type Conversion

Implicit type conversion, also known as **type coercion**, is the automatic conversion of a value from one data type to another by the JavaScript engine during program execution.

Features

- Performed automatically by the JavaScript engine
- Requires no explicit instruction from the programmer
- Occurs when different data types are used together in an expression
- Commonly applied during arithmetic and comparison operations

Syntax

value1 operator value2

Example

```
let result = "10" + 5;  
console.log(result);
```

Output

105

Here, the number 5 is automatically converted into a string, and string concatenation is performed.

Advantages

- 1) Reduces the amount of code required
- 2) Provides flexibility when working with different data types
- 3) Simplifies simple operations involving mixed data types

Limitations

- 1) May produce unexpected results
- 2) Can make program behavior difficult to predict
- 3) Increases the possibility of logical errors

Explicit Type Conversion

Explicit type conversion is the manual conversion of a value from one data type to another using built-in JavaScript conversion functions.

Features

- Performed under the control of the programmer
- Provides greater accuracy and predictability
- Improves readability and maintainability of code
- Commonly used when processing user input

Syntax

Function / Operator	Description
String(value)	Converts any data type (number, boolean, etc.) into a string explicitly.
value.toString()	Converts a value into string format, but does not work with null and undefined.
Number(value)	Converts a value into a number (integer or float depending on input).
parseInt(value)	Converts a string into an integer by removing decimal part.
parseFloat(value)	Converts a string into a floating-point (decimal) number.
Boolean(value)	Converts a value into true or false based on truthy/falsy rules.
+value (Unary plus)	Converts a string or other type into a number quickly.

Example

```
let age = "20";  
let numAge = Number(age);  
console.log(numAge);  
console.log(typeof numAge);
```

Output

20

number

The string "20" is explicitly converted into the numeric value 20 using the Number() function.

Advantages

- 1) Provides precise control over data conversion
- 2) Reduces unexpected behavior caused by automatic conversion
- 3) Improves program reliability and accuracy
- 4) Makes code easier to understand and maintain

Limitations

- 1) Requires additional coding effort
- 2) Incorrect conversion methods may produce errors
- 3) The programmer must understand the appropriate conversion function to use

Input and Output in JavaScript

- Input and Output (I/O) operations are essential components of any programming language.
- In JavaScript, input operations allow users to provide data to a program, while output operations enable the program to display information to users.
- These operations facilitate communication between the user and the application, making web pages interactive and dynamic.

Input in JavaScript

- Input is the process of receiving data from the user or another source during the execution of a program.
- Input allows users to interact with the application by providing information such as names, ages, addresses, passwords, and other required data.

Methods of Taking Input in JavaScript

1. Using prompt() Method

The prompt() method displays a dialog box that requests input from the user. The entered value is returned as a string.

Syntax

```
variableName = prompt("Message");
```

Example

```
let name = prompt("Enter your name:");  
document.write("Name: " + name);
```

Output

```
Enter your name: Ram  
Name: Ram
```

2. Taking Input Using HTML Form Elements

HTML form controls such as text boxes, radio buttons, checkboxes, and dropdown lists can be used to collect user input.

Example

```
<!DOCTYPE html>  
<html>  
<body>  
<label>Enter Name:</label>  
<input type="text" id="name">  
<button onclick="displayName()">Submit</button>  
<script>  
function displayName()  
{  
    let name = document.getElementById("name").value;  
    document.write("Name: " + name);  
}  
</script>  
</body>  
</html>
```

Output in JavaScript

- Output is the process of displaying processed information or results to the user.
- Output helps users understand the result of program execution.

Methods of Displaying Output in JavaScript

1. document.write() Method in JavaScript

- The document.write() method is one of the simplest output methods in JavaScript.
- It is used to display text, numbers, variables, HTML elements, and other content directly on a web page.
- This method is commonly used by beginners to understand how JavaScript produces output.
- **Syntax: document.write(data);**

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>document.write </title>
</head>
<body>

<script>
  // Text
  document.write("Welcome to JavaScript");
  document.write("<br>");
  // Number
  document.write(100);
  document.write("<br>");
  // Variable
  let name = "Ram";
  document.write(name);
  document.write("<br>");
  // Arithmetic Expression
  let a = 10;
  let b = 20;
  document.write("Sum = " + (a + b));
  document.write("<br>");
  // Boolean Value
  document.write(true);
  document.write("<br>");
  // HTML Tag
  document.write("<h3>This is Heading</h3>");
</script>

</body>
</html>
```

Output

```
Welcome to JavaScript
100
Ram
Sum = 30
true

This is Heading
```

Advantages

1. Easy to use.
2. Displays output directly on the webpage.
3. Useful for learning JavaScript.

Limitations

1. Can overwrite webpage content after page loading.

2. Not recommended for modern web applications.

2. alert() Method in JavaScript

- The alert() method is a built-in JavaScript output method used to display a message in a popup dialog box.
- It is commonly used to provide information, warnings, and notifications to users.
- The alert box contains an **OK** button that must be clicked before the user can continue interacting with the webpage.
- It can display text, numbers, variables, Boolean values, and expressions.
- **Syntax:** alert(data);

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>alert()</title>
</head>
<body>
<script>
  // Text
  alert("Welcome to JavaScript");
  // Number
  alert(100);
  // Variable
  let name = "Ram";
  alert(name);
  // Arithmetic Expression
  let a = 10;
  let b = 20;
  alert("Sum = " + (a + b));
  // Boolean Value
  alert(true);
</script>
</body>
</html>
```

Output

```
Popup Message: Welcome to JavaScript
Popup Message: 100
Popup Message: Ram
Popup Message: Sum = 30
Popup Message: true
```

Advantages

1. Simple and easy to use.
2. Provides immediate feedback to users.
3. Useful for displaying warnings and notifications.
4. Requires minimal code.

Limitations

1. Interrupts user interaction until the dialog box is closed.
2. Not suitable for displaying large amounts of information.
3. Provides limited formatting options.
4. Excessive use may reduce user experience.

3. console.log() Method in JavaScript

- The console.log() method is a built-in JavaScript output method used to display messages in the browser's console.
- It is primarily used for testing, debugging, and monitoring program execution.
- The output is displayed in the browser's Developer Tools Console rather than on the webpage.
- It can display text, numbers, variables, Boolean values, arrays, objects, and expressions.
- **Syntax:** console.log(data);

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>console.log()</title>
</head>
<body>
<script>
  // Text
  console.log("Welcome to JavaScript");
  // Number
  console.log(100);
  // Variable
  let name = "Ram";
  console.log(name);
  // Arithmetic Expression
  let a = 10;
  let b = 20;
  console.log("Sum = " + (a + b));
  console.log(true);
</script>
</body>
</html>
```

Output (Browser Console)

```
Welcome to JavaScript
100
Ram
Sum = 30
true
```

Advantages

1. Useful for debugging programs.
2. Does not affect the appearance of the webpage.
3. Can display multiple types of data.
4. Helps programmers identify and fix errors.

Limitations

1. Output is visible only in the browser console.
2. End users generally cannot see the output.
3. Not suitable for displaying information directly to users.

4. innerHTML Property in JavaScript

- The innerHTML property is used to access or modify the HTML content of a specific HTML element.
- It allows JavaScript to dynamically change the content displayed on a web page.
- It can insert text, numbers, variables, and HTML tags inside an element.
- **Syntax:** document.getElementById("id").innerHTML = data;

Example

```
<!DOCTYPE html>
<html>
```



```

<head>
  <title>innerHTML Example</title>
</head>
<body>

<p id="result"></p>

<script>

  // Text
  document.getElementById("result").innerHTML = "Welcome to JavaScript";

</script>

</body>
</html>

```

Output

Welcome to JavaScript

Example (Multiple Types)

```

<!DOCTYPE html>
<html>
<head>
  <title>innerHTML </title>
</head>
<body>

<p id="p1 "></p>

<script>
  let name = "Ram";
  let a = 10;
  let b = 20;
  document.getElementById("p1").innerHTML =
    "Name: " + name + "<br>" +
    "Sum = " + (a + b);
</script>
</body>
</html>

```

Advantages

- Allows dynamic modification of webpage content.
- Supports insertion of HTML tags.
- Widely used in modern web development.
- Provides better control compared to document.write().

Limitations

1. Improper use may lead to security risks (e.g., HTML injection).
2. Requires proper selection of HTML elements.

3. Can become complex in large applications if not structured properly.

textContent Property in JavaScript

- The textContent property is used to set or return only the text content of an HTML element.
- It does not interpret or render HTML tags; it treats them as plain text.
- It is commonly used for displaying secure and simple text output in web pages.
- **Syntax:** document.getElementById("id").textContent = data;

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>textContent </title>
</head>
<body>

  <p id="demo"></p>
  <script>

    let name = "Ram";
    document.getElementById("demo").textContent = "Name: " + name;

  </script>

</body>
</html>
```

Output

Name: Ram

Example (Multiple Values)

```
<!DOCTYPE html>
<html>
<head>
  <title>textContent </title>
</head>
<body>

<p id="result"></p>

<script>

  let name = "Ram";
  let age = 20;

  document.getElementById("result").textContent =
    "Name: " + name + ", Age: " + age;

</script>

</body>
</html>
```

Advantages

1. Displays only plain text without HTML interpretation.
2. More secure compared to innerHTML.
3. Faster performance for text-only updates.
4. Simple and easy to use.

Limitations

1. Cannot render HTML tags.
2. Not suitable when formatting (like headings, bold text) is required.
3. Limited to plain text manipulation only.

Introduction to Functions in JavaScript

- JavaScript functions are fundamental building blocks used to structure and organize code in a logical and reusable manner.
- A function is a reusable block of code that performs a specific task when it is called.
- Functions help in dividing a large program into smaller and manageable parts, improving clarity and maintainability.
- They reduce code repetition by allowing the same code to be executed multiple times without rewriting it.
- A function executes only when it is invoked (called) in the program.
- Functions are widely used in JavaScript for tasks such as data processing, event handling, form validation, and dynamic content generation.
- Proper use of functions leads to modular programming, making applications easier to debug, test, and maintain.

Syntax of Function

```
function functionName()  
{  
    // statements  
}
```

Components of a Function

Function Keyword

- The function keyword is used to declare a function in JavaScript.
- It indicates the start of a function definition.

Function Name

- The function name is used to uniquely identify and call the function.
- It should be meaningful and represent the task performed by the function.

Parameters

- Parameters are variables defined in the function declaration.
- They act as input values passed to the function during execution.
- Parameters are optional.

Function Body

- The function body contains the set of executable statements.
- It is enclosed within curly braces { }.
- This section defines the actual logic of the function.

Return Statement

- The return statement is used to send a value back to the calling environment.
- It also terminates the execution of the function.
- It is optional depending on the requirement.

Function Call (Invocation)

- A function call is the process of executing a function.
- Without calling, the function definition alone does not execute.

- It may pass arguments to the function.

```
<!DOCTYPE html>
<html>
<head>
  <title>Function Components </title>
</head>
<body>

<script>

  // Function declaration (function keyword + name + parameters)
  function add(a, b)
  {
    // Function body (processing logic)
    let sum = a + b;

    // Return statement (sending result back)
    return sum;
  }

  // Function call (invocation with arguments)
  let result = add(10, 20);

  // Output using document.write()
  document.write("Sum = " + result);

</script>

</body>
</html>
```

Output

Sum = 30

Types of Functions in JavaScript

Function Without Parameters and Without Return Value

- This type of function does not accept any input (parameters).
- It also does not return any value.
- It is mainly used for displaying messages or performing simple tasks.

Syntax

```
function functionName()  
{  
    // statements  
}
```

Example

```
<!DOCTYPE html>  
<html>  
<body>  
  
<script>  
  
function showMessage()  
{  
    document.write("Hello Students");  
}  
  
showMessage();  
  
</script>  
  
</body>  
</html>
```

Output

Hello Students

Function With Parameters and Without Return Value

- This type of function accepts input values through parameters.
- It does not return any value.
- The result is directly displayed inside the function.

Syntax

```
function functionName(parameter1, parameter2)  
{  
    // statements  
}
```

Example

```
<!DOCTYPE html>
```

```
<html>
<body>

<script>

function greet(name)
{
    document.write("Welcome " + name);
}

greet("Ram");

</script>

</body>
</html>
```

Output

Welcome Ram

Function Without Parameters and With Return Value

- This type of function does not take any input.
- It returns a value after execution.
- The returned value is used outside the function.

Syntax

```
function functionName()
{
    return value;
}
```

Example

```
<!DOCTYPE html>
<html>
<body>
<script>
function getNumber()
{
    return 100;
}
let value = getNumber();
document.write(value);
</script>
</body>
</html>
```

Output 100

Function With Parameters and With Return Value

- This type of function accepts input values and returns a computed result.
- It is the most commonly used function type in real applications.

Syntax

```
function functionName(param1, param2)
{
    return result;
}
```

Example

```
<!DOCTYPE html>
<html>
<body>

<script>

function add(a, b)
{
    return a + b;
}

let result = add(10, 20);
document.write("Sum = " + result);

</script>

</body>
</html>
```

Output

Sum = 30

Advantages of Functions

1. Improves code reusability.
2. Reduces duplication of code.
3. Enhances modular programming.
4. Improves readability and structure.
5. Simplifies debugging and maintenance.

Limitations of Functions

1. Improper design may increase complexity.
2. Excessive function calls may slightly affect performance.
3. Large programs may become difficult to manage without proper structure.

Applications of Functions

1. Form validation in web pages.
2. Mathematical computations.
3. Event handling.

4. Data processing in web applications.
5. Dynamic content updates.

Recursive Function in JavaScript

- A recursive function is a function that calls itself repeatedly until a specified condition is satisfied.
- It is used to solve problems that can be divided into smaller sub-problems of the same type.
- Recursion is widely used in mathematical computations, tree structures, and algorithm design.

Syntax:

```
function functionName()  
{  
    if (base condition)  
    {  
        return result;  
    }  
    else  
    {  
        return functionName(); // recursive call  
    }  
}
```

Find factorial of a number using recursive function.

```
<!DOCTYPE html>  
<html>  
<body>  
<script>  
function factorial(n)  
{  
    if (n == 0 || n == 1)  
    {  
        return 1; // base condition  
    }  
    else  
    {  
        return n * factorial(n - 1); // recursive call  
    }  
}  
let result = factorial(5);  
document.write("Factorial = " + result);  
</script>  
</body>  
</html>
```

Output: Factorial = 120

Advantages of Recursive Function

1. Simplifies complex problems.
2. Reduces code length.
3. Useful for mathematical and tree-based problems.

4. Improves code readability for certain problems.

Limitations of Recursive Function

1. Uses more memory due to function stack.
2. Can lead to infinite recursion if base condition is missing.
3. Slower compared to iterative solutions in some cases.

Date Object in JavaScript

- The **Date Object** in JavaScript is a built-in object used to handle **date and time values**.
- It allows developers to create, retrieve, manipulate, and display date-time information based on the system clock.
- Internally, JavaScript represents date and time as **milliseconds elapsed from 1 January 1970 (Unix Epoch Time)**.
- The Date Object is widely used in web applications such as:
 - Digital clock systems
 - Attendance systems
 - Event scheduling systems
 - Online booking systems
 - Age calculation systems
 - Logging and auditing systems

Ways to Create a Date Object in JavaScript

1. Create Current Date and Time

This method creates a Date object with the current system date and time.

Example

```
let d = new Date();  
console.log(d);
```

Output

Sun Jun 21 2026 10:30:45 GMT+0545

2. Create Date Using Year, Month, Day

This method is used to create a specific date.

Example

```
let d = new Date(2026, 5, 21);  
console.log(d);
```

Output

Sun Jun 21 2026

Month starts from 0 (0 = January, 5 = June)

3. Create Date Using Year, Month, Day, Time

This method is used when date and time both are needed.

Example

```
let d = new Date(2026, 5, 21, 10, 30, 45);  
console.log(d);
```

Output

Sun Jun 21 2026 10:30:45

Create Date Using String

This method creates a date from a text format.

Example

```
let d = new Date("June 21, 2026");
```

```
console.log(d);
```

Output

Sun Jun 21 2026

Create Date Using Milliseconds

JavaScript stores time in milliseconds from 1 Jan 1970.

Example

```
let d = new Date(0);  
console.log(d);
```

Output

Thu Jan 01 1970 00:00:00 GMT+0000

Getter Methods (Retrieving Date Components)

Getter methods are used to extract specific parts of a date object.

Method	Description
getDate()	Returns day of month (1–31)
getMonth()	Returns month (0–11)
getFullYear()	Returns full year
getDay()	Returns weekday (0–6)
getHours()	Returns hour (0–23)
getMinutes()	Returns minutes (0–59)
getSeconds()	Returns seconds (0–59)

Example (Getter Methods)

```
let now = new Date();  
  
console.log("Date:", now.getDate());  
console.log("Month:", now.getMonth() + 1);  
console.log("Year:", now.getFullYear());  
console.log("Day:", now.getDay());  
console.log("Time:", now.getHours() + ":" + now.getMinutes() + ":" + now.getSeconds());
```

Output

Date: 21
Month: 6
Year: 2026
Day: 0
Time: 10:30:45

Setter Methods (Modifying Date Values)

Setter methods are used to change existing date values.

Method	Description
setDate()	Sets day of month
setMonth()	Sets month
setFullYear()	Sets year
setHours()	Sets hour
setMinutes()	Sets minutes

setSeconds()	Sets seconds
--------------	--------------

Example (Setter Methods)

```
let d = new Date();

console.log("Original Date:", d);

d.setFullYear(2030);
d.setMonth(11);
d.setDate(25);
d.setHours(8);
d.setMinutes(30);

console.log("Modified Date:", d);
```

Output

```
Original Date: Sun Jun 21 2026 10:30:45 GMT+0545
Modified Date: Fri Dec 25 2030 08:30:45 GMT+0545
```

Applications of Date Object

1. Real-time clock systems
2. Attendance management systems
3. Event scheduling systems
4. Online booking systems
5. Age calculation systems
6. Logging and tracking systems

Advantages

1. Built-in and easy to use
2. Supports both retrieval and modification
3. Essential for real-time applications
4. Standardized time representation
5. Reduces manual time-handling complexity

Limitations

1. Month index starts from 0
2. Requires manual formatting for user-friendly display
3. Time zone handling may require additional logic

Math Functions in JavaScript

- The **Math object** in JavaScript is a built-in object used to perform **mathematical operations**.
- It provides many useful functions for calculations such as rounding numbers, finding square roots, powers, and generating random numbers.
- The Math object is **static**, so we do not need to create an object using new.

Common Math Functions

Category	Function	Description	Example	Output
Rounding	Math.round(x)	Rounds to nearest integer	Math.round(4.6)	5
Rounding	Math.ceil(x)	Rounds upward	Math.ceil(4.2)	5
Rounding	Math.floor(x)	Rounds downward	Math.floor(4.9)	4
Rounding	Math.trunc(x)	Removes decimal part	Math.trunc(4.9)	4
Power	Math.pow(x,y)	x raised to power y	Math.pow(2,3)	8
Root	Math.sqrt(x)	Square root of x	Math.sqrt(16)	4
Absolute	Math.abs(x)	Returns positive value	Math.abs(-10)	10
Max	Math.max(a,b,...)	Returns largest value	Math.max(10,50,30)	50
Min	Math.min(a,b,...)	Returns smallest value	Math.min(10,50,30)	10
Random	Math.random()	Random number (0–1)	Math.random()	0.57 (example)
Random Range	Math.floor(Math.random()*100)+1	Random 1–100	same	42 (example)
Constant	Math.PI	Value of π	Math.PI	3.14159
Constant	Math.E	Euler's number	Math.E	2.718

Example Program

```

<script>
let a = 16;
let b = -9;

console.log("Number a:", a);
console.log("Number b:", b);

console.log("\n--- Rounding Functions ---");
console.log("Round(4.6):", Math.round(4.6));
console.log("Ceil(4.2):", Math.ceil(4.2));
console.log("Floor(4.9):", Math.floor(4.9));
console.log("Trunc(4.9):", Math.trunc(4.9));

console.log("\n--- Power and Root ---");
console.log("Power (2^3):", Math.pow(2, 3));
console.log("Square Root of a:", Math.sqrt(a));

console.log("\n--- Absolute Value ---");
console.log("Absolute of b:", Math.abs(b));

console.log("\n--- Maximum and Minimum ---");
console.log("Max:", Math.max(10, 50, 30, 5));
console.log("Min:", Math.min(10, 50, 30, 5));

console.log("\n--- Random Numbers ---");
console.log("Random (0-1):", Math.random());
console.log("Random (1-100):", Math.floor(Math.random() * 100) + 1);
</script>

```


ES6 Fundamentals

- **ES6 (ECMAScript 2015)** is an important version update of JavaScript that introduced many new features and improvements. It was designed to make JavaScript code more **modern, readable, and easier to manage**.
- Before ES6, JavaScript had limitations in writing structured and reusable code. ES6 solved many of these issues by introducing new syntax and programming features.
- ES6 does not replace JavaScript; instead, it **enhances JavaScript with new features and syntax improvements**.
- ES6 was introduced to:
 - Improve the structure of JavaScript code
 - Reduce complexity in programming
 - Support modern programming practices
 - Make code more readable and maintainable
 - Support large-scale application development

Importance of ES6

ES6 is widely used in modern web development because:

- It is the standard version used in most browsers and frameworks
- It supports object-oriented and functional programming styles
- It is used in frameworks like React, Angular, and Node.js
- It helps developers write cleaner and efficient code

Key Features

1. let and const

let and const to improve variable handling in JavaScript. Both are **block-scoped**, meaning they are valid only within { }.

- let is used when the value can be changed
- const is used when the value should remain constant

Example

```
let a = 10;  
a = 20;  
const b = 30;  
console.log(a);  
console.log(b);
```

Output

```
20  
30
```

2. Arrow Functions

- Arrow functions provide a **short and clean syntax** for writing functions.
- They reduce code length and improve readability.
- They are commonly used in modern JavaScript, especially in callbacks.

Example

```
const add = (x, y) => x + y;  
console.log(add(5, 3));
```

Output

8

3. Template Literals

Template literals allow strings to be written using backticks (`). They support variable insertion using `\${}`.

- Easier string formatting
- Supports multi-line strings

Example

```
let name = "Ram";  
  
console.log(`Hello ${name}, welcome to ES6`);
```

Output

Hello Ram, welcome to ES6

4. Default Parameters

Default parameters assign a default value to function parameters when no argument is passed.

Example

```
function greet(name = "Guest") {  
  console.log("Hello " + name);  
}  
  
greet();  
greet("Sita");
```

Output

Hello Guest
Hello Sita

5. Destructuring

Destructuring allows extraction of values from arrays or objects into separate variables in a simple way.

Example

```
let arr = [10, 20, 30];

let [a, b, c] = arr;

console.log(a);
console.log(b);
console.log(c);
```

Output

```
10
20
30
```

6. Spread Operator

- The spread operator (...) is used to expand elements of an array or object.
- It is commonly used for copying and merging data structures.

Example

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1, 4, 5];

console.log(arr2);
```

Output

```
[1, 2, 3, 4, 5]
```

7. Rest Parameter

The rest parameter collects multiple function arguments into a single array. It is useful when the number of inputs is not fixed.

Example

```
function sum(...nums) {
  let total = 0;

  for (let n of nums) {
    total += n;
  }

  console.log(total);
}

sum(10, 20, 30);
```

Output

```
60
```

8. Classes

ES6 introduces **class syntax** to support object-oriented programming. A class is a blueprint for creating objects.

It includes:

- Constructor (initialization)
- Methods (functions inside class)

Example

```
class Student {  
  constructor(name) {  
    this.name = name;  
  }  
  
  show() {  
    console.log(this.name);  
  }  
}  
  
let s1 = new Student("RAM");  
s1.show();
```

Output

RAM

9. for...of Loop

The for...of loop is used to iterate through iterable objects such as arrays. It is simpler than traditional loop structures.

Example

```
let arr = [10, 20, 30];  
  
for (let val of arr) {  
  console.log(val);  
}
```

Output

10
20
30

Constructor in JavaScript

- A constructor is a special method in a JavaScript class that is automatically executed when a new object is created. It is used to initialize object properties with the values provided during object creation.
- The constructor method is named constructor() and is called automatically when the new keyword is used.

Features of Constructor

Automatic Execution	Called automatically when an object is created.
Object Initialization	Initializes object properties with values.
Special Method	Uses the reserved keyword constructor.
Uses this Keyword	Assigns values to the current object.
Invoked with new	Executes when an object is created using new.
Single Constructor	A class can have only one constructor method.
Code Reusability	Allows creation of multiple objects with different values.

Syntax

```
class ClassName {  
    constructor(parameters) {  
        // initialization code  
    }  
}
```

Example

```
class Student {  
    constructor(name, age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
let s1 = new Student("Karan", 28);
```

```
console.log("Name:", s1.name);
```

```
console.log("Age:", s1.age);
```

Output

Name: Karan

Age: 28

Advantages of ES6

1. Improved code readability and maintainability
2. Better variable handling using let and const
3. Cleaner and shorter syntax with arrow functions and template literals
4. Simplified data handling using destructuring, spread, and rest operators
5. Modular programming support using import and export

Array in JavaScript

- An array is a non-primitive data type in JavaScript that is used to store multiple values in a single variable.
- It allows related data to be grouped together and accessed efficiently using index numbers.
- Arrays are widely used in web applications to manage collections of data such as student records, product lists, employee details, marks, and user information.
- Without arrays, a programmer would need to create separate variables for each value, making programs difficult to manage and maintain. Arrays provide a convenient and organized way to handle large amounts of data.
- An array is an ordered collection of values stored in a single variable. Each value stored in the array is called an element, and each element is identified by an index number.

Features of Arrays

- Stores multiple values in a single variable.
- Elements are stored in an ordered sequence.
- Index numbering starts from zero.
- Supports dynamic resizing.
- Can store different types of data.
- Provides numerous built-in methods for manipulation.
- Simplifies data processing through loops.

Need for Arrays

Consider storing marks of five students.

Without Array

```
let mark1 = 80;  
let mark2 = 75;  
let mark3 = 90;  
let mark4 = 85;  
let mark5 = 88;
```

Managing a large number of variables becomes difficult.

With Array

```
let marks = [80, 75, 90, 85, 88];
```

Using an array makes the program more organized and efficient.

Array Declaration and Initialization

Array Literal Method

The array literal method is the simplest and most commonly used technique.

Compiled BY :Er.karan Dev Bhatt

Syntax

```
let arrayName = [value1, value2, value3];
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];
```

Array Constructor Method

Arrays can also be created using the Array constructor.

Syntax

```
let arrayName = new Array(value1, value2, value3);
```

Example

```
let fruits = new Array("Apple", "Mango", "Orange");
```

Array Indexing

- Each element in an array has a position called an index.
- Index numbering starts from 0.

Example

```
let fruits = ["Apple", "Mango", "Orange"];
```

Element	Index
Apple	0
Mango	1
Orange	2

Accessing Array Elements

Accessing array elements means retrieving values stored in the array using index numbers.

Syntax

```
arrayName[index];
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];
```

```
document.write(fruits[0] + "<br>");  
document.write(fruits[1] + "<br>");  
document.write(fruits[2]);
```

Output

```
Apple  
Mango  
Orange
```

Modifying Array Elements

Array elements can be changed by assigning a new value to a specific index.

Syntax

```
arrayName[index] = newValue;
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];
fruits[1] = "Banana";
document.write(fruits);
```

Output

Apple,Banana,Orange

Array Length Property

The length property returns the total number of elements present in an array.

Syntax

```
arrayName.length
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];

document.write(fruits.length);
```

Output

3

Traversing Arrays

Traversing means accessing each element of an array one by one.

Traversing Using for Loop

Example

```
let fruits = ["Apple", "Mango", "Orange"];

for(let i = 0; i < fruits.length; i++)
{
    document.write(fruits[i] + "<br>");
}
```

Traversing Using for...of Loop

Example

```
let fruits = ["Apple", "Mango", "Orange"];

for(let fruit of fruits)
{
    document.write(fruit + "<br>");
}
```

Array Methods

push() Method

Adds one or more elements to the end of an array.

Syntax

```
arrayName.push(element);
```

Example

```
let fruits = ["Apple", "Mango"];  
fruits.push("Orange");  
document.write(fruits);
```

Output

Apple,Mango,Orange

pop() Method

Removes the last element from an array.

Syntax

```
arrayName.pop();
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];  
  
fruits.pop();  
  
document.write(fruits);
```

Output

Apple,Mango

unshift() Method

Adds one or more elements to the beginning of an array.

Syntax

```
arrayName.unshift(element);
```

Example

```
let fruits = ["Mango", "Orange"];  
  
fruits.unshift("Apple");  
  
document.write(fruits);
```

Output

Apple,Mango,Orange

shift() Method

Removes the first element from an array.

Syntax

```
arrayName.shift();
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];
fruits.shift();
document.write(fruits);
```

Output

Mango,Orange

indexOf() Method

Returns the index position of a specified element.

Syntax

```
arrayName.indexOf(element);
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];

document.write(fruits.indexOf("Mango"));
```

Output

1

includes() Method

Determines whether a specified element exists in the array.

Syntax

```
arrayName.includes(element);
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];

document.write(fruits.includes("Orange"));
```

Output

true

sort() Method

Sorts the elements of an array.

Syntax

```
arrayName.sort();
```

Example

```
let numbers = [30, 10, 20];

numbers.sort();
```

```
document.write(numbers);
```

Output

10,20,30

reverse() Method

Reverses the order of elements in an array.

Syntax

```
arrayName.reverse();
```

Example

```
let fruits = ["Apple", "Mango", "Orange"];
```

```
fruits.reverse();
```

```
document.write(fruits);
```

Output

Orange,Mango,Apple

Types of Arrays

1. One-Dimensional Array

A one-dimensional array stores elements in a single row.

Example

```
let numbers = [10, 20, 30, 40];
```

Program to Input and Display Array Elements

```
<!DOCTYPE html>
<html>
<head>
  <title>One-Dimensional Array</title>
</head>
<body>

<script>
  let arr = [];
  let n = parseInt(prompt("Enter the number of elements:"));

  for(let i = 0; i < n; i++)
  {
    arr[i] = prompt("Enter element " + (i + 1) + ":");
  }

  document.write("<h3>Array Elements</h3>");

  for(let i = 0; i < n; i++)
  {
    document.write(arr[i] + "<br>");
  }
</script>

</body>
```

```
</html>
```

Program to Find Sum of Array Elements

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Sum of Array Elements</title>
```

```
</head>
```

```
<body>
```

```
<script>
```

```
  let arr = [];
```

```
  let n = parseInt(prompt("Enter the number of elements:"));
```

```
  let sum = 0;
```

```
  for(let i = 0; i < n; i++)
```

```
  {
```

```
    arr[i] = parseInt(prompt("Enter element " + (i + 1) + ":"));
```

```
    sum += arr[i];
```

```
  }
```

```
  document.write("Array Elements: <br>");
```

```
  for(let i = 0; i < n; i++)
```

```
  {
```

```
    document.write(arr[i] + "<br>");
```

```
  }
```

```
  document.write("<br>Sum = " + sum);
```

```
</script>
```

```
</body>
```

```
</html>
```

Program to Find Largest Element in an Array

```
<!DOCTYPE html>
<html>
<head>
  <title>Largest Element</title>
</head>
<body>

<script>
  let arr = [];
  let n = parseInt(prompt("Enter the number of elements:"));

  for(let i = 0; i < n; i++)
  {
    arr[i] = parseInt(prompt("Enter element " + (i + 1) + ":"));
  }

  let largest = arr[0];

  for(let i = 1; i < n; i++)
  {
    if(arr[i] > largest)
    {
      largest = arr[i];
    }
  }

  document.write("Largest Element = " + largest);
</script>

</body>
</html>
```

Program to Find Smallest Element in an Array

```
<!DOCTYPE html>
<html>
<head>
  <title>Smallest Element</title>
</head>
<body>

<script>
  let arr = [];
  let n = parseInt(prompt("Enter the number of elements:"));

  for(let i = 0; i < n; i++)
  {
    arr[i] = parseInt(prompt("Enter element " + (i + 1) + ":"));
  }
}
```

```

let smallest = arr[0];

for(let i = 1; i < n; i++)
{
    if(arr[i] < smallest)
    {
        smallest = arr[i];
    }
}

document.write("Smallest Element = " + smallest);
</script>

</body>
</html>

```

Program to Search an Element in an Array

```

<!DOCTYPE html>
<html>
<head>
    <title>Search Element</title>
</head>
<body>

<script>
    let arr = [];
    let n = parseInt(prompt("Enter the number of elements:"));

    for(let i = 0; i < n; i++)
    {
        arr[i] = prompt("Enter element " + (i + 1) + ":");
    }

    let search = prompt("Enter element to search:");
    let found = false;

    for(let i = 0; i < n; i++)
    {
        if(arr[i] === search)
        {
            found = true;
            break;
        }
    }

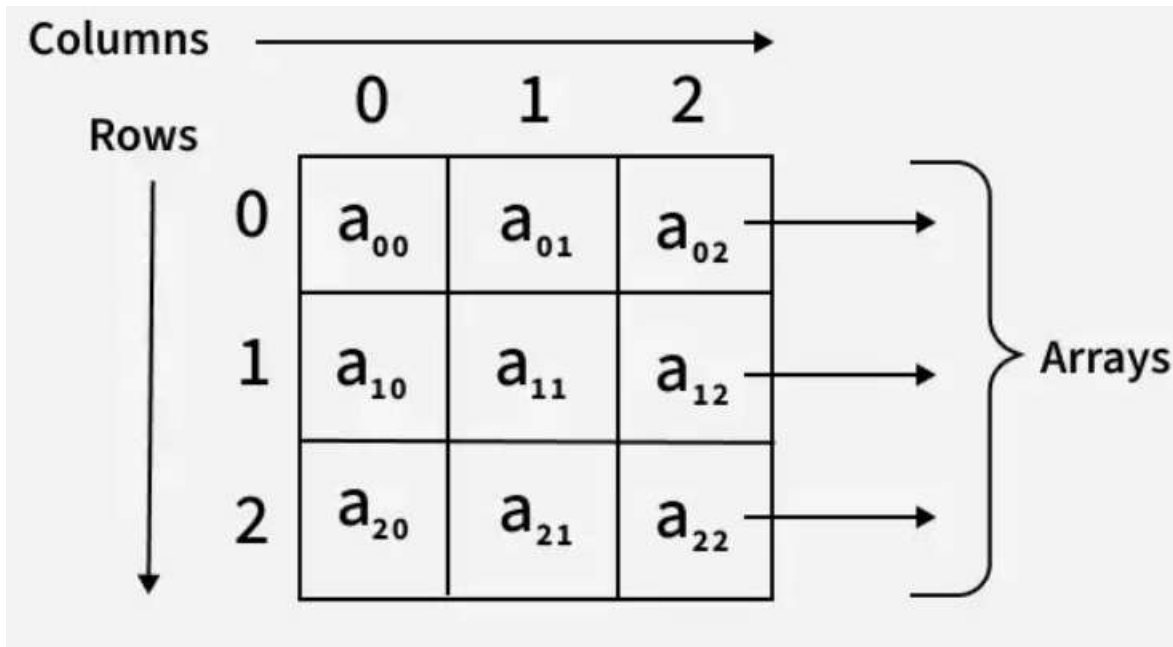
    if(found)
        document.write("Element Found");
    else
        document.write("Element Not Found");
</script>

```

</body>
</html>

Two-Dimensional Array

A two-dimensional array contains multiple arrays as elements.



Syntax:

```
let arrayName = [  
    [value1, value2, value3],  
    [value4, value5, value6],  
    [value7, value8, value9]  
];
```

Example

```
let marks = [  
    [80, 75, 90],  
    [70, 85, 88]  
];  
document.write(marks[1][2]);
```

Output

88

Input and Display Matrix Elements

```
<!DOCTYPE html>
<html>
<head>
  <title>Display Matrix</title>
</head>
<body>

<script>
  let rows = parseInt(prompt("Enter number of rows:"));
  let cols = parseInt(prompt("Enter number of columns:"));

  let matrix = [];

  for(let i = 0; i < rows; i++)
  {
    matrix[i] = [];

    for(let j = 0; j < cols; j++)
    {
      matrix[i][j] = parseInt(prompt("Enter element:"));
    }
  }

  document.write("Matrix Elements:<br>");

  for(let i = 0; i < rows; i++)
  {
    for(let j = 0; j < cols; j++)
    {
      document.write(matrix[i][j] + " ");
    }
    document.write("<br>");
  }
</script>

</body>
</html>
```

Find Sum of All Matrix Elements

```
<!DOCTYPE html>
<html>
<head>
  <title>Sum of Matrix Elements</title>
</head>
<body>

<script>
  let rows = parseInt(prompt("Enter number of rows:"));
  let cols = parseInt(prompt("Enter number of columns:"));
```

```

let matrix = [];
let sum = 0;

for(let i = 0; i < rows; i++)
{
    matrix[i] = [];

    for(let j = 0; j < cols; j++)
    {
        matrix[i][j] = parseInt(prompt("Enter element:"));
        sum += matrix[i][j];
    }
}

document.write("Sum of Elements = " + sum);
</script>

```

```

</body>
</html>

```

Sample Output

Sum of Elements = 210

Find Largest Element in a Matrix

```

<!DOCTYPE html>
<html>
<head>
    <title>Largest Element</title>
</head>
<body>

<script>
    let rows = parseInt(prompt("Enter number of rows:"));
    let cols = parseInt(prompt("Enter number of columns:"));

    let matrix = [];

    for(let i = 0; i < rows; i++)
    {
        matrix[i] = [];

        for(let j = 0; j < cols; j++)
        {
            matrix[i][j] = parseInt(prompt("Enter element:"));
        }
    }

    let largest = matrix[0][0];

    for(let i = 0; i < rows; i++)
    {

```

```

        for(let j = 0; j < cols; j++)
        {
            if(matrix[i][j] > largest)
            {
                largest = matrix[i][j];
            }
        }
    }
}

document.write("Largest Element = " + largest);
</script>

</body>
</html>

```

Advantages of Arrays

1. Stores multiple values efficiently.
2. Reduces the number of variables.
3. Simplifies data processing.
4. Supports dynamic storage.
5. Provides powerful built-in methods.
6. Enhances program readability.

Limitations of Arrays

1. Large arrays may consume more memory.
2. Access requires correct index values.
3. Searching elements in large arrays may take time.
4. Complex data structures may require objects instead of arrays.

Write a JavaScript program to store and display information of students using arrays.

```

<!DOCTYPE html>
<html>
<head>
    <title>Student Information</title>
</head>
<body>

<script>
    let names = ["Ram", "Sita", "Hari"];
    let ages = [20, 21, 22];
    let addresses = ["Pokhara", "Kathmandu", "Butwal"];

    document.write("<h3>Student Information</h3>");

    for(let i = 0; i < names.length; i++)
    {
        document.write("Student " + (i + 1) + "<br>");
        document.write("Name: " + names[i] + "<br>");
        document.write("Age: " + ages[i] + "<br>");
        document.write("Address: " + addresses[i] + "<br><br>");
    }

```



```
</script>
```

```
</body>
```

```
</html>
```

Output

Student 1

Name: Ram

Age: 20

Address: Pokhara

Student 2

Name: Sita

Age: 21

Address: Kathmandu

Student 3

Name: Hari

Age: 22

Address: Butwal

Alternative

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Student Information</title>
```

```
</head>
```

```
<body>
```

```
<script>
```

```
  let students = [  
    {name: "Ram", age: 20, address: "Pokhara"},  
    {name: "Sita", age: 21, address: "Kathmandu"},  
    {name: "Hari", age: 22, address: "Butwal"}  
  ];
```

```
  document.write("<h3>Student Information</h3>");
```

```
  for(let i = 0; i < students.length; i++)
```

```
  {  
    document.write("Student " + (i + 1) + "<br>");  
    document.write("Name: " + students[i].name + "<br>");  
    document.write("Age: " + students[i].age + "<br>");  
    document.write("Address: " + students[i].address + "<br><br>");  
  }
```

```
</script>
```

```
</body>
```

```
</html>
```

Sample Output

Student 1

Name: Ram
Age: 20
Address: Pokhara

Student 2
Name: Sita
Age: 21
Address: Kathmandu

Student 3
Name: Hari
Age: 22
Address: Butwal

Element Access in JavaScript

Element Access in JavaScript refers to the process of selecting and retrieving HTML elements from a web page so that they can be manipulated dynamically. JavaScript interacts with HTML documents through the **Document Object Model (DOM)**, which represents every element in a webpage as an object.

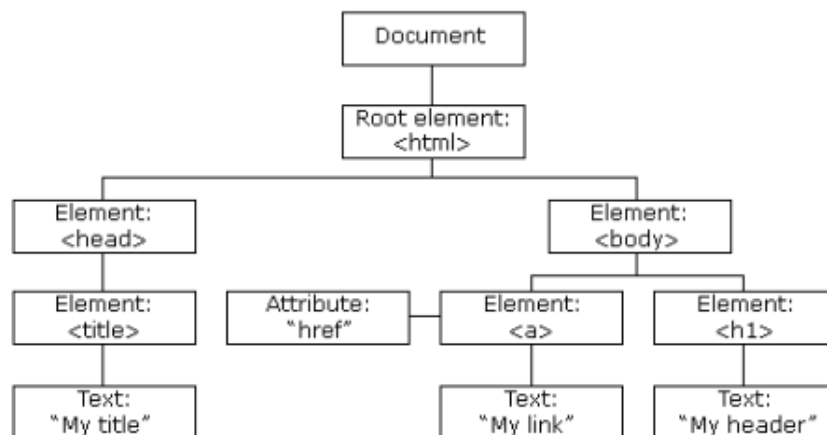
By accessing elements, JavaScript can:

- Read and modify content.
- Change styles and attributes.
- Handle user input.
- Validate forms.
- Create interactive web applications.
- Respond to user actions dynamically.

Element access is one of the most fundamental concepts in JavaScript and serves as the basis for DOM manipulation.

Document Object Model (DOM)

The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a tree structure.



Each HTML tag becomes an object (node) that JavaScript can access and manipulate.

Example HTML Document

```
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>Hello Students</p>
</body>
</html>
```

JavaScript uses DOM methods to access these nodes.

Methods of Accessing Elements

A. Accessing Elements by ID

The `getElementById()` method selects an element using its unique ID.

Characteristics

- Returns a single element.
- ID must be unique.
- Most commonly used method.

Syntax

```
document.getElementById("id");
```

Example

```
<p id="message">Welcome to JavaScript</p>
```

```
<script>
let element =
document.getElementById("message");

console.log(element.textContent);
</script>
```

Output

Welcome to JavaScript

Accessing Elements by Class Name

The `getElementsByClassName()` method selects all elements having the specified class name.

Characteristics

- Returns an `HTMLCollection`.
- Multiple elements can share the same class name.
- Elements are accessed using indexes.

Syntax

```
document.getElementsByClassName("className");
```

Example

```
<p class="note">First Note</p>
<p class="note">Second Note</p>

<script>
let notes =
document.getElementsByClassName("note");

console.log(notes[0].textContent);
</script>
```

Output

First Note

Accessing Elements by Tag Name

The `getElementsByTagName()` method selects elements based on their HTML tag name.

Characteristics

- Returns all matching tags.
- Produces an HTMLCollection.
- Useful for processing multiple similar elements.

Syntax

```
document.getElementsByTagName("tagname");
```

Example

```
<h1>Heading One</h1>
```

```
<h1>Heading Two</h1>
```

```
<script>
```

```
let headings =
```

```
document.getElementsByTagName("h1");
```

```
console.log(headings[1].textContent);
```

```
</script>
```

Output

Heading Two

Accessing Elements Using `querySelector()`

The `querySelector()` method selects the first element that matches a CSS selector.

Syntax

```
document.querySelector("selector");
```

Example

```
<p class="info">Information</p>
```

```
<script>
```

```
let data =
```

```
document.querySelector(".info");
```

```
console.log(data.textContent);
```

```
</script>
```

Output

Information

Characteristics

- Uses CSS selector syntax.
- Returns only the first matching element.
- Highly flexible.

Accessing Elements Using `querySelectorAll()`

The `querySelectorAll()` method selects all elements that match a CSS selector.

Syntax

```
document.querySelectorAll("selector");
```

Example

```
<p class="item">Item One</p>
<p class="item">Item Two</p>

<script>
let items =
document.querySelectorAll(".item");

console.log(items[1].textContent);
</script>
```

Output

Item Two

Characteristics

- Returns a NodeList.
- Supports CSS selectors.
- Suitable for accessing multiple elements.

Comparison of Element Access Methods

Method	Returns	Selection Basis
getElementById()	Single Element	ID
getElementsByClassName()	HTMLCollection	Class Name
getElementsByTagName()	HTMLCollection	Tag Name
querySelector()	First Matching Element	CSS Selector
querySelectorAll()	NodeList	CSS Selector

Reading Element Content

After selecting an element, JavaScript can retrieve its content.

Using innerHTML

The innerHTML property retrieves the HTML content inside an element.

Example

```
<p id="demo">Welcome</p>

<script>
let content =document.getElementById("demo").innerHTML;

console.log(content);
</script>
```

Output

Welcome

Using textContent

The textContent property retrieves only textual content.

Example

```
<p id="msg">Hello Students</p>
```

```
<script>
let text =
document.getElementById("msg").textContent;

console.log(text);
</script>
```

Output

Hello Students

Difference Between innerHTML and textContent

innerHTML	textContent
Returns HTML and text	Returns only text
Recognizes HTML tags	Ignores HTML tags
Can insert HTML elements	Cannot insert HTML elements
Slightly slower	Faster

Example

```
<p id="demo">
<b>Hello</b> Students
</p>
console.log(
document.getElementById("demo").innerHTML
);
Output:
```

```
<b>Hello</b> Students
console.log(
document.getElementById("demo").textContent
);
```

Output:

Hello Students

Reading User Input

The .value property is used to retrieve data entered into form controls.

Example

```
<input type="text" id="username">

<button onclick="showData()">
Submit
</button>

<script>
function showData()
{
let name =
document.getElementById("username").value;
```

```
    alert(name);  
}  
</script>
```

Modifying Element Content

JavaScript can dynamically change webpage content.

Using innerHTML

Example

```
<p id="demo">Old Content</p>  
  
<script>  
document.getElementById("demo").innerHTML =  
"New Content";  
</script>
```

Output

New Content

Using textContent

Example

```
<p id="message">Welcome</p>  
  
<script>  
document.getElementById("message").textContent =  
"Hello Students";  
</script>
```

Output

Hello Students

Modifying Styles Through Element Access

The style property allows JavaScript to modify CSS properties dynamically.

Example

```
<p id="text">JavaScript Example</p>  
  
<script>  
document.getElementById("text").style.color =  
"red";  
  
document.getElementById("text").style.fontSize =  
"24px";  
</script>
```

Example

```
<!DOCTYPE html>  
<html>  
<body>
```



```
<h2 id="title">Welcome</h2>

<button onclick="changeText()">
Click Me
</button>

<script>
function changeText()
{
    document.getElementById("title")
    .textContent =
    "Welcome to JavaScript";
}
</script>

</body>
</html>
```

Advantages of Element Access

- Enables dynamic content updates.
- Supports form validation.
- Improves user interaction.
- Allows modification of styles and attributes.
- Facilitates event handling.
- Creates responsive web applications.
- Enhances user experience.

Event Handling in JavaScript

JavaScript is a client-side scripting language used to create dynamic and interactive web applications. One of the most important features of JavaScript is its ability to respond to user actions and browser activities through **Event Handling**.

When users interact with a web page by clicking a button, entering text, selecting an option, or submitting a form, various events are generated. JavaScript can detect these events and execute specific instructions in response. This mechanism is known as **Event Handling**.

Event handling forms the foundation of interactive web development and enables developers to create responsive and user-friendly applications.

An **Event** is an action or occurrence detected by the web browser.

Events may be generated by:

- User interactions
- Browser activities
- System-generated actions

When an event occurs, JavaScript can execute predefined code to respond appropriately.

Examples of Events

User Action	Generated Event
Clicking a button	click
Double-clicking a button	dblclick
Typing in a textbox	keyup
Selecting a dropdown option	change
Submitting a form	submit
Loading a web page	load
Moving mouse over an element	mouseover

Event Handling is the process of detecting events and executing appropriate JavaScript code when those events occur.

In simple terms:

Event Handling is the mechanism through which JavaScript responds to user actions and browser-generated activities.

Importance of Event Handling

4. Event handling is essential in modern web development because it:
5. Users can interact with web pages dynamically.
6. Applications can respond immediately to user actions.
7. Users receive instant responses without reloading the page.
8. Input can be validated before submission.
9. Many operations can be performed on the client side.
10. Dynamic content can be updated without refreshing the entire page.

Event-Driven Programming

JavaScript follows an **Event-Driven Programming Model**.

In this model:

1. The program waits for events.
2. When an event occurs, an event handler executes.
3. The application performs the required action.

An **Event Handler** is a JavaScript function that executes when a specific event occurs.

Example

```
function showMessage()  
{  
    alert("Welcome");  
}
```

The above function becomes an event handler when attached to an event.

Methods of Event Handling

JavaScript provides several methods for handling events.

Inline Event Handling

JavaScript code is written directly inside an HTML element.

Example

```
<button onclick="alert('Hello')">  
Click Me  
</button>
```

Advantages

1. Easy to implement.

Disadvantages

1. Mixes HTML and JavaScript.
2. Difficult to maintain in large projects.

Traditional Event Handling

An event is assigned through a JavaScript property.

Example

```
document.getElementById("btn").onclick =  
function()  
{  
    alert("Button Clicked");  
};
```

Limitation

Only one event handler can be assigned to an event.

addEventListener() Method

The addEventListener() method attaches an event handler to an HTML element.

It is the most commonly used and recommended method in modern JavaScript.

Syntax

```
element.addEventListener(  
    "event", function()  
    {  
        // code  
    }  
);
```

Components of Syntax

Component	Description
element	HTML element
addEventListener()	Method used to attach an event
event	Event type
function()	Event handler function

Example

```
document.getElementById("btn")
.addEventListener("click",
function()
{
    alert("Button Clicked");
});
```

Advantages of addEventListener()

- Multiple event handlers can be attached.
- Cleaner and more organized code.
- Separation of HTML and JavaScript.
- Better maintainability.

Classification of Events

JavaScript events can be categorized into the following groups:

1. Mouse Events
2. Keyboard Events
3. Form Events
4. Window Events

Event Category	Event	Description
Mouse Events	click	Occurs when the user clicks an element
	dblclick	Occurs when the user double-clicks an element
	mouseover	Occurs when the mouse pointer enters an element
	mouseout	Occurs when the mouse pointer leaves an element
	mousedown	Occurs when a mouse button is pressed
	mouseup	Occurs when a mouse button is released
	mousemove	Occurs when the mouse pointer moves over an element
Keyboard Events	keydown	Occurs when a key is pressed
	keyup	Occurs when a key is released
	keypress	Occurs when a key is pressed (legacy event)
Form Events	focus	Occurs when an input field receives focus
	blur	Occurs when an input field loses focus
	change	Occurs when the value of a form element changes
	submit	Occurs when a form is submitted
	reset	Occurs when a form is reset
Window Events	input	Occurs when the value of an input field changes
	load	Occurs when the web page has fully loaded
	resize	Occurs when the browser window is resized
	scroll	Occurs when the user scrolls the page
	unload	Occurs when the user leaves the page
	beforeunload	Occurs before the page is unloaded

Example :Mouse Events

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Mouse Events</title>
</head>
<body>

<h2>Mouse Events</h2>
<div id="box"
  style="width:250px;height:100px;
  background-color:lightblue;
  text-align:center;
  line-height:100px;">
  Move Mouse Here
</div>

<p id="output"></p>

<script>

let box = document.getElementById("box");

box.addEventListener("click", function()
{
  document.getElementById("output").innerHTML =
  "Click Event Occurred";
});

box.addEventListener("dblclick", function()
{
  document.getElementById("output").innerHTML =
  "Double Click Event Occurred";
});

box.addEventListener("mouseover", function()
{
  document.getElementById("output").innerHTML =
  "Mouse Over Event Occurred";
});

box.addEventListener("mouseout", function()
{
  document.getElementById("output").innerHTML =
  "Mouse Out Event Occurred";
});

</script>

</body>
</html>
```

Example :Keyboard Events

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Keyboard Events </title>
</head>
<body>

<h2>Keyboard Events</h2>

<input type="text"
  id="txtName"
  placeholder="Type Here">

<p id="output"></p>

<script>

let txt =
document.getElementById("txtName");

txt.addEventListener("keydown", function()
{
  document.getElementById("output").innerHTML =
    "Key Down Event";
});

txt.addEventListener("keyup", function()
{
  document.getElementById("output").innerHTML =
    "Key Up Event";
});

</script>

</body>
</html>
```

Example :Form Events

```
<!DOCTYPE html>
<html>
<head>
  <title>Form Events Example</title>
</head>
<body>

<h2>Form Events</h2>

<form id="myForm">

<input type="text"
  id="name"
```

```
placeholder="Enter Name">

<br><br>

<select id="course">
  <option>Select Course</option>
  <option>Computer Engineering</option>
  <option>Civil Engineering</option>
</select>

<br><br>

<button type="submit">
Submit
</button>

</form>

<p id="output"></p>

<script>

document.getElementById("name")
.addEventListener("focus", function()
{
  document.getElementById("output").innerHTML =
  "Focus Event";
});

document.getElementById("name")
.addEventListener("blur", function()
{
  document.getElementById("output").innerHTML =
  "Blur Event";
});

document.getElementById("course")
.addEventListener("change", function()
{
  document.getElementById("output").innerHTML =
  "Change Event";
});

document.getElementById("myForm")
.addEventListener("submit", function(event)
{
  event.preventDefault();

  document.getElementById("output").innerHTML =
  "Submit Event";
});
```



```
</script>
```

```
</body>
```

```
</html>
```

Example :Window Events

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Window Events Example</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Window Events</h2>
```

```
  <p id="output"></p>
```

```
  <script>
```

```
    window.addEventListener("load", function()
```

```
    {
```

```
      document.getElementById("output").innerHTML =  
      "Page Loaded Successfully";
```

```
    });
```

```
    window.addEventListener("resize", function()
```

```
    {
```

```
      document.getElementById("output").innerHTML =  
      "Window Resized";
```

```
    });
```

```
    window.addEventListener("scroll", function()
```

```
    {
```

```
      document.getElementById("output").innerHTML =  
      "Page Scrolled";
```

```
    });
```

```
  </script>
```

```
  <div style="height:1500px;"></div>
```

```
</body>
```

```
</html>
```

Advantages of Event Handling

1. Enables user interaction.
2. Provides immediate feedback.
3. Supports dynamic web pages.
4. Improves usability.

5. Facilitates form validation.
6. Reduces page reloads.
7. Enhances overall application responsiveness.

Error Handling in JavaScript

During program execution, unexpected situations may occur due to invalid input, programming mistakes, unavailable resources, or incorrect operations. These situations are known as **errors**.

If errors are not handled properly, a program may stop executing or produce incorrect results. JavaScript provides an **Error Handling Mechanism** that allows developers to detect, manage, and respond to errors gracefully.

Error handling improves the reliability, stability, and usability of web applications.

Error

An **Error** is an abnormal condition that occurs during program execution and prevents a program from performing its intended task.

Example

```
console.log(studentName);
```

Output

ReferenceError: studentName is not defined

Error Handling

Error Handling is the process of detecting, catching, and managing errors during program execution so that the program can continue running properly.

Purpose of Error Handling

Prevent Crashes	Stops the program from terminating unexpectedly
Improve Reliability	Makes applications more stable
Better User Experience	Displays meaningful messages
Easy Debugging	Helps identify and fix problems
Data Validation	Detects invalid user input

Types of Errors in JavaScript

Error Type	Description	Example
Syntax Error	Violation of JavaScript syntax rules	Missing bracket
Reference Error	Using an undefined variable	console.log(x)
Type Error	Invalid operation on a data type	10.toUpperCase()
Range Error	Value outside allowable range	toPrecision(500)
Logical Error	Program executes but produces incorrect output	Wrong formula

Syntax Error

A Syntax Error occurs when JavaScript code violates language grammar rules.

Example

```
if(true
{
  console.log("Hello");
}
```

Output

SyntaxError

Reference Error

A Reference Error occurs when an undeclared variable is accessed.

Example

```
console.log(name);
```

Output

ReferenceError: name is not defined

Type Error

A Type Error occurs when an operation is performed on an inappropriate data type.

Example

```
let num = 10;
```

```
num.toUpperCase();
```

Output

TypeError

Range Error

A Range Error occurs when a value exceeds the acceptable range.

Example

```
let num = 10;
```

```
num.toPrecision(500);
```

Output

RangeError

Logical Error

A Logical Error occurs when a program runs successfully but produces incorrect output.

Example

```
let total = 10 + 20 * 2;
```

```
console.log(total);
```

Output

50

Expected Output

60

Error Handling Statements

JavaScript provides four major statements for handling errors.

Statement	Purpose
try	Contains code that may generate an error
catch	Handles errors generated in try block
finally	Executes whether an error occurs or not
throw	Creates custom errors

try Statement

The try block contains code that may potentially generate an error.

Syntax

```
try
{
    // risky code
}
```

Example

```
try
{
    console.log(x);
}
```

catch Statement

The catch block executes when an error occurs inside the try block.

Syntax

```
try
{
    // code
}
catch(error)
{
    // handle error
}
```

Example

```
try
{
    console.log(x);
}
catch(error)
{
    console.log("Error Found");
}
```

Output

Error Found

Error Object

When an error occurs, JavaScript automatically creates an **Error Object**.

Common Properties

Property	Description
error.name	Returns error type
error.message	Returns detailed error message

Example

```
try
{
    console.log(x);
}
catch(error)
{
    console.log(error.name);
    console.log(error.message);
}
```

Output

ReferenceError
x is not defined

finally Statement

The finally block executes regardless of whether an error occurs.

Syntax

```
try
{
}
catch(error)
{
}
finally
{
}
```

Example

```
try
{
    console.log(x);
}
catch(error)
{
    console.log("Error Found");
}
finally
{
    console.log("Program Finished");
}
```

Output

```
Error Found
Program Finished
```

throw Statement

The throw statement is used to generate custom errors.

Syntax

```
throw "Error Message";
```

Example

```
let age = 15;

try
{
    if(age < 18)
    {
        throw "Age must be 18 or above";
    }
}
catch(error)
{
    console.log(error);
}
```

Output

```
Age must be 18 or above
```

Example: Age Validation

```
function checkAge()
{
    let age = document.getElementById("age").value;

    try
    {
        if(age == "")
        {
            throw "Age cannot be empty";
        }

        if(isNaN(age))
        {
            throw "Age must be numeric";
        }

        if(age < 18)
        {
            throw "Age must be 18 or above";
        }

        document.getElementById("result").innerHTML =
        "Eligible";
    }

    catch(error)
    {
        document.getElementById("result").innerHTML =
        error;
    }

    finally
    {
        console.log("Validation Completed");
    }
}
```

Regular Expressions (RegEx) in JavaScript

A **Regular Expression (RegEx)** is a sequence of characters that defines a search pattern. In JavaScript, Regular Expressions are used to perform pattern matching on strings. They are commonly used for searching, validating, extracting, and replacing text data.

Regular Expressions play an important role in web development, particularly in **form validation**, where user input such as names, email addresses, phone numbers, and passwords must be checked before processing.

A **Regular Expression** is a pattern used to match character combinations in a string. It enables developers to determine whether a particular text follows a specified format.

Example

Suppose a website requires users to enter a valid email address:

student@gmail.com

A Regular Expression can verify whether the entered email follows the correct structure before the form is submitted.

Applications of Regular Expressions

Regular Expressions are widely used in the following areas:

1. Form validation
2. Email validation
3. Phone number validation
4. Password validation
5. Searching text
6. Replacing text
7. Data extraction
8. Input filtering

Creating Regular Expressions in JavaScript

JavaScript provides two methods for creating Regular Expressions.

1. Literal Notation

```
let pattern = /hello/;
```

This is the most commonly used method because it is simple and efficient.

2. RegExp Constructor

```
let pattern = new RegExp("hello");
```

This method is useful when the pattern needs to be generated dynamically.

Structure of a Regular Expression

The general syntax of a Regular Expression is:

`/pattern/flags`

Example

```
let pattern = /javascript/i;
```

Regular Expression Flags

Flags modify the behavior of a Regular Expression.

Flag	Description
g	Performs a global search
i	Ignores letter case
m	Enables multiline matching
s	Allows the dot operator to match new lines
u	Enables Unicode matching

Example

```
let text = "JavaScript javascript";
```

```
console.log(text.match(/javascript/gi));
```

Output

```
["JavaScript", "javascript"]
```

The g flag finds all matches, while the i flag ignores uppercase and lowercase differences.

Common Regular Expression Methods

JavaScript provides several methods for working with Regular Expressions. The most commonly used methods are:

Method	Purpose
test()	Checks whether a pattern exists in a string
exec()	Returns detailed information about a match
match()	Retrieves matching text from a string
matchAll()	Retrieves all matches from a string
search()	Finds the position of the first match
replace()	Replaces matching text with new text
replaceAll()	Replaces all matching text
split()	Divides a string into an array based on a pattern

1. test() Method

The test() method checks whether a string contains a specified pattern. It returns a Boolean value indicating whether the match exists.

Syntax

```
pattern.test(string)
```

Example

```
let pattern = /JavaScript/;
```

```
console.log(
  pattern.test("I am learning JavaScript")
);
```

Output

```
true
```

The string contains the word "JavaScript", so the method returns true.

2. exec() Method

The exec() method searches a string for a specified pattern and returns detailed information about the first match.

Syntax

```
pattern.exec(string)
```

Example

```
let pattern = /Java/;
```

```
console.log(
  pattern.exec("JavaScript")
);
```

Output

```
["Java"]
```

The method finds the first occurrence of "Java" and returns it as an array.

3. match() Method

The match() method searches a string and returns the matching text.

Syntax

```
string.match(pattern)
```

Example

```
let text = "JavaScript Tutorial";
console.log(
  text.match(/JavaScript/)
);
```

Output

```
["JavaScript"]
```

The matching word "JavaScript" is returned in an array.

4. matchAll() Method

The matchAll() method returns all occurrences of a pattern found in a string.

Syntax

```
string.matchAll(pattern)
```

Example

```
let text = "cat bat rat";
```

```
let result =
  text.matchAll(/at/g);
```

```
for(let item of result)
{
  console.log(item[0]);
}
```

Output

```
at
at
at
```

The pattern at appears three times and all matches are returned.

5. search() Method

The search() method searches a string for a specified pattern and returns the index position of the first match.

Syntax

```
string.search(pattern)
```

Example

```
let text = "Learn JavaScript";
```

```
console.log(text.search(/JavaScript/));
```

Output

```
6
```

The word "JavaScript" begins at index position 6.

6. replace() Method

The replace() method replaces the first occurrence of matching text with new text.

Syntax

```
string.replace(pattern,replacement)
```

Example

```
let text =
"I like Java";
```

```
console.log(
text.replace(/Java/,"JavaScript")
);
```

Output

```
I like JavaScript
```

The word "Java" is replaced with "JavaScript".

7. replaceAll() Method

The replaceAll() method replaces all occurrences of matching text within a string.

Syntax

```
string.replaceAll(pattern,replacement)
```

Example

```
let text =
```

```
"cat cat cat";
```

```
console.log(  
  text.replaceAll("cat","dog")  
);
```

Output

```
dog dog dog
```

All occurrences of "cat" are replaced with "dog".

8. **split() Method**

The `split()` method divides a string into multiple parts based on a specified separator or pattern.

Syntax

```
string.split(pattern)
```

Example

```
let text =  
"HTML,CSS,JavaScript";
```

```
console.log(  
  text.split(/,/)  
);
```

Output

```
["HTML","CSS","JavaScript"]
```

Character Classes

Character classes represent predefined groups of characters.

Character Class	Description	Matches
\d	Digit character	0–9
\D	Non-digit character	Any character except 0–9
\w	Word character	A-Z, a-z, 0-9, _
\W	Non-word character	Special symbols
\s	Whitespace character	Space, tab, newline
\S	Non-whitespace character	Any character except whitespace
.	Any single character except newline	Any character

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Character Classes </title>
</head>
<body>

  <h2>JavaScript Character Classes </h2>

  <script>

    let text = "Everest123 @ Kathmandu";

    console.log("Original Text:", text);

    // \d : Digit Character
    console.log("\d (Digits):");
    console.log(text.match(/\d/g));

    // \D : Non-Digit Character
    console.log("\D (Non-Digits):");
    console.log(text.match(/\D/g));

    // \w : Word Character
    console.log("\w (Word Characters):");
    console.log(text.match(/\w/g));

    // \W : Non-Word Character
    console.log("\W (Non-Word Characters):");
    console.log(text.match(/\W/g));

    // \s : Whitespace Character
    console.log("\s (Whitespace Characters):");
    console.log(text.match(/\s/g));

    // \S : Non-Whitespace Character
    console.log("\S (Non-Whitespace Characters):");
    console.log(text.match(/\S/g));
```

Compiled BY :Er.karan Dev Bhatt

```
// . : Any Character
console.log(". (Any Character:");
console.log(text.match(/./g));
```

```
</script>
```

```
</body>
```

```
</html>
```

Quantifiers

- **Quantifiers** are special symbols used in Regular Expressions to specify **how many times a character, group, or character class should occur** in a string.
- Quantifiers make Regular Expressions more flexible by allowing patterns to match repeated characters or groups of characters.

Common Quantifiers

Quantifier	Meaning
*	Zero or more occurrences
+	One or more occurrences
?	Zero or one occurrence
{n}	Exactly n occurrences
{n,}	At least n occurrences

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Quantifiers Example</title>
</head>
<body>
```

```
<h2>Quantifiers Example</h2>
```

```
<script>
```

```
// * (Zero or More)
```

```
document.write("<b>* Quantifier:</b><br>");
document.write(/ab*/.test("abbb") + "<br><br>");
```

```
// + (One or More)
```

```
document.write("<b>+ Quantifier:</b><br>");
document.write(/ab+/.test("abbb") + "<br><br>");
```

```
// ? (Zero or One)
```

```
document.write("<b>? Quantifier:</b><br>");
document.write(/colou?r/.test("color") + "<br>");
document.write(/colou?r/.test("colour") + "<br><br>");
```

```
// {n} (Exactly n Times)
document.write("<b>{n} Quantifier:</b><br>");
document.write(/\d{4}/.test("2026") + "<br><br>");
```

```
// {n,} (At Least n Times)
document.write("<b>{n,} Quantifier:</b><br>");
document.write(/a{3,}/.test("aaaa") + "<br><br>");
```

```
// {n,m} (Between n and m Times)
document.write("<b>{n,m} Quantifier:</b><br>");
document.write(/\d{2,5}/.test("1234") + "<br>");
```

```
</script>
```

```
</body>
```

```
</html>
```

Anchors

An **Anchor** is a special symbol in a Regular Expression that matches a specific position within a string rather than matching actual characters.

Anchors are used to specify:

- The beginning of a string
- The end of a string
- The entire string

Anchors are important because they:

- Improve the accuracy of pattern matching.
- Ensure that input follows a predefined format.
- Prevent partial matches when complete matching is required.
- Simplify form validation.
- Enhance data integrity in web applications.
- JavaScript Regular Expressions provide two primary anchors:

Anchor	Name	Purpose
^	Beginning Anchor	Matches the beginning of a string
\$	Ending Anchor	Matches the end of a string

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Start (^) and End ($) Anchor Example</title>
</head>
<body>
<input type="text" id="code" placeholder="Enter EEC">
<button onclick="checkCode()">Check</button>
<p id="result"></p>
<script>
function checkCode() {
  let code = document.getElementById("code").value;

  // String must be exactly "EEC"
  let pattern = /^EEC$/;

  if (pattern.test(code)) {
    document.getElementById("result").innerHTML = "Valid";
  } else {
    document.getElementById("result").innerHTML = "Invalid";
  }
}
</script>
</body>
</html>
```

Advantages of Regular Expressions

1. Simplifies data validation.
2. Reduces programming complexity.
3. Improves data accuracy.
4. Enhances application security.
5. Provides efficient pattern matching.
6. Supports text searching and replacement operations.

Limitations of Regular Expressions

1. Complex patterns may be difficult to understand.
2. Long expressions can reduce code readability.
3. Debugging complex patterns can be challenging.

Form Validation Using Regular Expressions in JavaScript

Form Validation is the process of checking whether user input is correct, complete, and follows predefined rules before the form is submitted to the server.

Validation helps ensure that users enter valid information and prevents incorrect or incomplete data from being stored in the system.

JavaScript Regular Expressions (RegEx) are commonly used to perform client-side validation efficiently.

The main objectives of form validation are:

- Ensure data accuracy.
- Prevent invalid data entry.
- Improve application security.
- Enhance user experience.
- Reduce server-side processing.

Types of Validation

Validation Type	Purpose
Empty Field Validation	Checks whether a field is blank
Name Validation	Allows only letters and spaces
Email Validation	Verifies email format
Phone Validation	Verifies phone number format
Password Validation	Checks password strength
Confirm Password Validation	Ensures passwords match

Regular Expressions for Form Validation

Name Validation

The name validation pattern ensures that the input contains only alphabetic characters (A–Z, a–z) and spaces. Numbers and special characters are not allowed.

Regular Expression: `/^[A-Za-z ]+$/`

<code>^</code>	Indicates the beginning of the string.
<code>[A-Za-z]</code>	Allows uppercase letters (A–Z), lowercase letters (a–z), and spaces.
<code>+</code>	Requires one or more occurrences of the preceding character set.
<code>\$</code>	Indicates the end of the string.

Email Validation

The email validation pattern checks whether the input follows the standard email format:
username@domain.extension.

Regular Expression: `/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$/`

<code>^</code>	Indicates the beginning of the string.
<code>[a-zA-Z0-9._%+-]+</code>	Username part containing letters, digits, and permitted special characters (., _, %, +, -).

@	Mandatory separator between username and domain.
[a-zA-Z0-9.-]+	Domain name containing letters, digits, dots, and hyphens.
\.	Represents a literal dot (.).
[A-Za-z]{2,}	Domain extension containing at least two alphabetic characters.
\$	Indicates the end of the string.

Phone Number Validation

The phone number validation pattern verifies Nepal mobile numbers that start with 98 or 97 and contain a total of 10 digits.

Regular Expression: `/^(98|97)\d{8}$/`

^	Indicates the beginning of the string.
(98 97)	Match either 98 or 97
\d	Represents any digit from 0 to 9.
{8}	Requires exactly 10 digits after the prefix.
\$	Indicates the end of the string.

Password Validation

The password validation pattern enforces a strong password policy. The password must contain at least one lowercase letter, one uppercase letter, one digit, one special character, and be at least 8 characters long.

Regular Expression: `/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/`

^	Indicates the beginning of the string.
(?=.*[a-z])	Requires at least one lowercase letter.
(?=.*[A-Z])	Requires at least one uppercase letter.
(?=.*\d)	Requires at least one digit.
(?=.*[@\$!%*?&])	Requires at least one special character from the specified set.
.	Represents any character.
{8,}	Requires a minimum length of 8 characters.
\$	Indicates the end of the string.

Date Validation (YYYY/MM/DD)

The date validation pattern verifies whether the entered date follows the YYYY/MM/DD format. where:

- **YYYY** represents a 4-digit year.
- **MM** represents a month from 01 to 12.
- **DD** represents a day from 01 to 32.
- The date components are separated by forward slashes (/).

Regular Expression: `/^\d{4}\(0[1-9]|1[0-2])\((0[1-9]|12|[0-9]|3[0-2])\)$/`

Year (YYYY)	<code>\d{4}</code>	Year must contain exactly 4 digits (e.g., 2083).
Month (MM)	<code>(0[1-9] 1[0-2])</code>	Month must be between 01 and 12.
Day (DD)	<code>(0[1-9] 12 [0-9] 3[0-2])</code>	Day must be between 01 and 32.

Visa Card Validation

A Visa card number must:

- Start with the digit **4**.
- Contain either **13 digits** or **16 digits** in total.
- Consist only of numeric digits.

Regular Expression `/^4[0-9]{12}(?:[0-9]{3})?$/`

Card Prefix	<code>4</code>	Card must start with 4.
Base Digits	<code>[0-9]{12}</code>	Requires 12 digits after the prefix.
Optional Digits	<code>(?:[0-9]{3})?</code>	Allows 3 extra digits, making a 16-digit card number.

Form Validation Example

```
<!DOCTYPE html>
<html>
<head>
<title>Form Validation</title>

<style>
body{
    font-family: Arial;
}

label{
    font-weight: bold;
}

input{
    width: 300px;
    padding: 8px;
    margin-bottom: 10px;
}

span{
    color: red;
}

button{
    padding: 10px 20px;
}
</style>

</head>
<body>

<h2>Student Registration Form</h2>

<form onsubmit="return validateForm()">

<label>Name *</label><br>
<input type="text" id="name">
<span id="nameError"></span>
<br>

<label>Email *</label><br>
<input type="text" id="email">
<span id="emailError"></span>
<br>

<label>Phone Number *</label><br>
<input type="text" id="phone">
<span id="phoneError"></span>
<br>
```

```

<label>Password *</label><br>
<input type="password" id="password">
<span id="passwordError"></span>
<br>

<label>Confirm Password *</label><br>
<input type="password" id="confirmPassword">
<span id="confirmError"></span>
<br>

<button type="submit">Submit</button>

</form>

<script>

function validateForm()
{
    let name =document.getElementById("name").value;

    let email = document.getElementById("email").value;

    let phone = document.getElementById("phone").value;

    let password = document.getElementById("password").value;

    let confirmPassword =document.getElementById("confirmPassword").value;

    document.getElementById("nameError").innerHTML = "";
    document.getElementById("emailError").innerHTML = "";
    document.getElementById("phoneError").innerHTML = "";
    document.getElementById("passwordError").innerHTML = "";
    document.getElementById("confirmError").innerHTML = "";

    let namePattern = /^[A-Za-z ]+$/;

    let emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}$/;

    let phonePattern = /^(98|97)\d{10}$/;

    let passwordPattern = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/;

    if(name == "")
    {
        document.getElementById("nameError").innerHTML =
        "Name is required";
        return false;
    }

    if(!namePattern.test(name))

```

```
{
  document.getElementById("nameError").innerHTML =
    "Enter valid name";
  return false;
}

if(!emailPattern.test(email))
{
  document.getElementById("emailError").innerHTML =
    "Enter valid email";
  return false;
}

if(!phonePattern.test(phone))
{
  document.getElementById("phoneError").innerHTML =
    "Enter valid phone number";
  return false;
}

if(!passwordPattern.test(password))
{
  document.getElementById("passwordError").innerHTML =
    "Password must contain uppercase, lowercase, digit and special character";
  return false;
}

if(password !== confirmPassword)
{
  document.getElementById("confirmError").innerHTML =
    "Passwords do not match";
  return false;
}

alert("Form Submitted Successfully");

return true;
}

</script>

</body>
</html>
```

Advantages of Form Validation

1. Improves data accuracy.
2. Prevents invalid data entry.
3. Enhances application security.
4. Reduces server workload.

5. Improves user experience.

Synchronous Programming in JavaScript

- JavaScript is a high-level, interpreted programming language that executes code in a sequential manner by default. This execution model is known as **synchronous programming**.
- In a synchronous environment, each statement is executed one after another, and a task must be completed before the next task begins execution.
- Synchronous programming provides a simple and predictable flow of control because program statements are executed in the exact order in which they appear. This execution model is suitable for operations that complete quickly, such as mathematical calculations, data processing, and input validation.
- **Synchronous programming** is a programming model in which program instructions are executed sequentially, and each operation must complete before the next operation begins execution.
- In synchronous execution, JavaScript waits for the current statement to finish before moving to the next statement.
- In a synchronous program:
 1. The first statement is executed.
 2. The program waits until the statement completes.
 3. The next statement is executed.
 4. This process continues until all statements have been processed.

Since JavaScript executes one task at a time, no subsequent operation can begin until the current operation is completed.

Characteristics

1. **Sequential Execution** – Statements are executed one after another in the order they appear.
2. **Blocking Nature** – The execution of subsequent statements is blocked until the current task is completed.
3. **Single-Threaded Execution** – JavaScript executes one task at a time in the main execution thread.
4. **Predictable Flow of Control** – Program execution follows a fixed and easily understandable sequence.
5. **Task Dependency** – Each task depends on the completion of the previous task before execution can proceed.
6. **Immediate Execution** – Statements are executed immediately when encountered by the JavaScript engine.
7. **Simple Program Structure** – Code is generally easier to write, understand, and maintain.
8. **Easy Debugging** – Errors can be identified and traced more easily due to the straightforward execution flow.
9. **Suitable for Short Operations** – Ideal for computations and tasks that complete quickly.
10. **Deterministic Execution Order** – The output is produced in the exact order in which statements are executed.
11. **No Concurrent Processing** – Multiple tasks cannot execute simultaneously within the main thread.
12. **Resource Waiting** – The program remains idle while waiting for long-running operations to complete.

Working of Synchronous Programming in JavaScript

- Program execution begins with the first statement.
- JavaScript executes statements sequentially in the order they appear.
- Each statement must complete before the next statement begins execution.
- The JavaScript engine waits for the current task to finish before proceeding to subsequent tasks.
- Function calls and statements are managed through the **Call Stack**.
- Only one operation is executed at a time.
- The execution flow remains linear and predictable throughout program execution.
- Long-running operations block the execution of subsequent statements until completion.
- The process continues until all statements have been executed and the Call Stack becomes empty.

Example1

```
console.log("Program Started");
```

```
console.log("Processing Data");
```

```
console.log("Program Ended");
```

Output

Program Started

Processing Data

Program Ended

Example with Functions

```
function displayMessage() {
```

```
    console.log("Welcome to JavaScript");
```

```
}
```

```
console.log("Start");
```

```
displayMessage();
```

```
console.log("End");
```

Output

Start

Welcome to JavaScript

End

Blocking Behavior in Synchronous Programming

- One of the most important characteristics of synchronous programming is its blocking nature.
- When a task requires a long time to complete, all subsequent statements must wait.

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Blocking Behavior Demo</title>
</head>
<body>

  <h2>Blocking Behavior in Synchronous JavaScript</h2>

  <button onclick="longTask()">
    Start Long Task
  </button>

  <button onclick="showMessage()">
    Click Me While Processing
  </button>

  <p id="output"></p>

  <script>

    function longTask() {

      document.getElementById("output").innerHTML =
        "Processing started...<br>";

      // Simulate heavy processing
      for(let i = 0; i < 5000000000; i++) {
      }

      document.getElementById("output").innerHTML +=
        "Processing completed.";
    }

    function showMessage() {

      alert("Second button clicked!");
    }

  </script>

</body>
</html>
```

Role of the Call Stack

- JavaScript uses a data structure known as the **Call Stack** to manage synchronous execution.
- The Call Stack follows the **Last In, First Out (LIFO)** principle.

Working Process

1. Functions are added to the stack when called.
2. The function at the top of the stack executes.
3. After completion, the function is removed from the stack.
4. The next function begins execution.

Example

```
function first() {  
    console.log("First");  
}  
  
function second() {  
    console.log("Second");  
}  
  
first();  
second();
```

Advantages of Synchronous Programming

1. **Simple to Understand** -Program execution follows a straightforward sequential flow.
2. **Easy to Debug** - Errors can be easily identified due to predictable execution order.
3. **Predictable Execution** -Statements execute in the exact order they are written.
4. **Suitable for Small Tasks** - Effective for quick computations and simple operations.
5. **Less Complex Logic** -Does not require asynchronous constructs such as callbacks or promises.
6. **Deterministic Results** - Produces consistent and expected outputs.

Disadvantages of Synchronous Programming

1. **Blocking Execution** -Subsequent tasks must wait for the current task to complete.
2. **Reduced Responsiveness** - Applications may become unresponsive during lengthy operations.
3. **Poor Performance for Long Tasks** -Time-consuming operations delay program execution.
4. **Inefficient Resource Utilization** -The program remains idle while waiting for operations to finish.
5. **Limited Scalability** -Not suitable for highly interactive and large-scale applications.
6. **Poor User Experience** -Users may experience delays and interface freezing.

Applications of Synchronous Programming

1. **Mathematical Calculations**
2. **Input Validation**
3. **String Manipulation**
4. **Array Processing**
5. **Data Formatting**
6. **Operations Requiring Strict Execution Order**

Asynchronous Programming in JavaScript

- Asynchronous programming allows JavaScript to execute time-consuming tasks without blocking the main program flow.
- Long-running operations can execute in the background while other statements continue to run.
- It improves application responsiveness and performance.
- JavaScript does not wait for an asynchronous task to complete before executing subsequent statements.
- It is commonly used for:
 1. API requests
 2. Server communication
 3. Database access
 4. File operations
 5. Data retrieval
- Asynchronous programming is the foundation of:
 1. AJAX
 2. XMLHttpRequest (XHR)
 3. Fetch API
 4. Callbacks
 5. Promises
 6. Async/Await

“Asynchronous programming is a programming model in which time-consuming operations are executed independently of the main program flow, allowing other tasks to continue without waiting for those operations to complete.”

Need for Asynchronous Programming

- In traditional synchronous execution, each statement must complete before the next statement begins. While this approach is simple, it becomes inefficient when operations require significant processing time, such operations include:
 - ✓ Fetching data from a web server
 - ✓ Accessing databases
 - ✓ Uploading and downloading files
 - ✓ Reading large files
 - ✓ Processing user requests
 - ✓ Communicating with external APIs
- If JavaScript executed these operations synchronously, the entire application would stop responding until the operation completed.
- Asynchronous programming addresses these challenges by allowing long-running tasks to execute without blocking the main execution thread.

Reasons for Using Asynchronous Programming

1. **Prevents Blocking:** Long-running operations do not prevent other instructions from executing.
2. **Improves User Experience:** Users can continue interacting with the application while background operations are processing.

3. **Enhances Application Performance:** The application remains responsive even during lengthy operations.
4. **Supports Real-Time Processing:** Applications can process live data updates efficiently.
5. **Efficient Resource Utilization:** The processor can perform other tasks instead of remaining idle while waiting for external resources.
6. **Supports Modern Web Applications:** Most contemporary web applications depend on asynchronous communication for dynamic content updates.

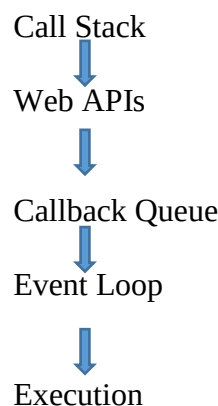
Characteristics of Asynchronous Programming

1. **Non-Blocking Execution:** Long-running tasks do not block the execution of other statements.
2. **Concurrent Task Handling:** Multiple operations can be initiated and managed efficiently.
3. **Event-Driven Nature:** Execution is often triggered by events such as clicks, timers, or server responses.
4. **Background Processing:** Time-consuming tasks execute in the background while the program continues running.
5. **Improved Responsiveness:** Applications remain interactive during lengthy operations.
6. **Use of Callbacks:** Callback functions can be executed after an asynchronous task completes.
7. **Support for Promises and Async/Await:** Modern techniques simplify asynchronous programming and improve code readability.
8. **Support for Real-Time Applications:** Commonly used in chat systems, live notifications, online gaming, and data streaming services.

Working Mechanism of Asynchronous Programming

1. **Execution of Synchronous Code:** JavaScript begins by executing normal statements through the Call Stack.
2. **Delegation of Asynchronous Tasks:** Time-consuming operations such as timers, API requests, and event handling are delegated to Web APIs provided by the browser.
3. **Continued Program Execution:** JavaScript continues executing the remaining statements without waiting for the asynchronous task to complete.
4. **Task Completion:** Once the asynchronous operation finishes, its callback function is placed in the Callback Queue.
5. **Event Loop Processing:** The Event Loop continuously monitors the Call Stack and Callback Queue.
6. **Callback Execution:** When the Call Stack becomes empty, the Event Loop moves the callback from the Callback Queue to the Call Stack for execution.
7. **Program Completion:** The callback executes, and the asynchronous task is completed.

Execution Flow



Example of Asynchronous Programming

```
console.log("Program Started");

setTimeout(function() {

    console.log("Data Loaded");

}, 2000);

console.log("Program Finished");
```

Output

Program Started
Program Finished
Data Loaded

Advantages of Asynchronous Programming

1. **Non-Blocking Execution:** Allows other tasks to continue while a long-running operation is being processed.
2. **Improved Responsiveness:** Keeps applications interactive and responsive.
3. **Better Performance:** Reduces waiting time and improves efficiency.
4. **Efficient Resource Utilization:** Enables the system to perform other tasks while waiting for external resources.
5. **Enhanced User Experience:** Prevents applications from becoming unresponsive.

Disadvantages of Asynchronous Programming

1. **Increased Complexity:** More difficult to understand and implement than synchronous programming.
2. **Difficult Debugging:** Execution flow can be harder to trace.
3. **Complex Error Handling:** Errors may be more challenging to manage.
4. **Callback Hell:** Excessive nesting of callbacks can reduce readability.
5. **Learning Curve:** Requires knowledge of callbacks, promises, and async/await.

Applications of Asynchronous Programming

1. **API Requests:** Retrieving data from web servers.
2. **AJAX Applications:** Updating webpage content without reloading.
3. **Database Operations:** Accessing and managing database records.
4. **File Handling:** Uploading, downloading, reading, and writing files.
5. **Real-Time Applications:** Chat systems, live notifications, and online gaming.
6. **Media Streaming:** Audio and video streaming services.
7. **E-Commerce Systems:** Processing orders, payments, and inventory updates.

Common Techniques for Asynchronous Programming

JavaScript provides several techniques for implementing asynchronous behavior.

Technique	Description
Callbacks	Functions executed after task completion
Promises	Objects representing future results

Async/Await	Simplified syntax for asynchronous programming
XMLHttpRequest (XHR)	Traditional method for server communication
Fetch API	Modern approach for HTTP requests
Event Listeners	Respond to user and system events

1.Callbacks in JavaScript

- A **callback** is a function that is passed as an argument to another function and is executed after a particular task has been completed. Callbacks are one of the fundamental techniques used in JavaScript to handle asynchronous operations.
- JavaScript frequently performs tasks that take time to complete, such as fetching data from a server, reading files, processing user input, or waiting for a timer. Instead of blocking the execution of the program while waiting for these operations to finish, JavaScript uses callbacks to specify what should happen when the operation is completed.

“A **callback function** is a function that is passed as an argument to another function and is invoked after the completion of a specific task or operation.”

Characteristics of Callbacks

- Functions can be passed as arguments.
- Executed after a task is completed.
- Commonly used in asynchronous programming.
- Help manage delayed operations.
- Frequently used in event handling.

Need for Callback Functions

- To handle asynchronous operations.
- To execute code after a task is completed.
- To maintain the correct execution order.
- To support event-driven programming.
- To improve code flexibility and reusability.

Syntax

```
function mainFunction(callback) {

    // Perform task

    callback();

}
```

Basic Example

```
function greet(name, callback) {

    console.log("Hello, " + name);

    callback();

}
```

```
}  
  
function sayBye() {  
    console.log("Goodbye!");  
}  
  
greet("Ajay", sayBye);  
Output
```

Hello, Ajay
Goodbye!

Working of Callback Functions

JavaScript executes code sequentially. However, some operations require time to complete. Callbacks allow JavaScript to execute a function after a task has finished.

Working Process

1. A callback function is passed to another function.
2. The main function starts execution.
3. A task is performed.
4. After the task completes, the callback function is invoked.
5. The callback performs the required action.

Callbacks for Asynchronous Execution

```
console.log("Start");  
  
setTimeout(function () {  
    console.log("Inside setTimeout");  
}, 2000);  
  
console.log("End");  
Output
```

Start
End
Inside setTimeout

Explanation

- setTimeout() is an asynchronous function.
- The timer starts and JavaScript continues execution.
- "End" is displayed immediately.
- After 2 seconds, the callback function executes.
- "Inside setTimeout" is displayed.

Problems with Callback Functions

Although callbacks are useful, they also introduce certain challenges.

1. Callback Hell

When callbacks are nested deeply, the code becomes difficult to read and maintain.

Example

```
step1() => {  
  step2() => {  
    step3() => {  
      console.log(  
        "All steps completed"  
      );  
    };  
  };  
};  
};  
};
```

Problems of Callback Hell

1. **Reduced Readability:** Deep nesting makes code difficult to understand.
2. **Difficult Debugging:** Errors are harder to locate and fix.
3. **Poor Maintainability:** Updating the code requires navigating multiple nested levels.
4. **Complex Error Handling:** Error management becomes repetitive and complicated.
5. **Code Duplication:** Similar callback logic may be repeated.

Solutions to Callback Hell

Modern JavaScript provides better alternatives:

1. **Promises:** Improve readability through chaining.
2. **Async/Await:** Provides a synchronous-like coding style.
3. **Modular Functions:** Break large tasks into smaller reusable functions.

2. Error Handling Issues

Managing errors across multiple callbacks can become complicated.

Example

```
function divide(a, b, callback) {  
  if(b === 0) {  
    callback(  
      new Error("Cannot divide by zero"),
```

Compiled BY :Er.karan Dev Bhatt


```

        null
    );

}
else {

    callback(
        null,
        a / b
    );

}

}

```

Problems

- Error handling code becomes repetitive.
- Multiple nested callbacks increase complexity.
- Difficult to trace error sources.

Advantages of Callback Functions

1. **Supports Asynchronous Programming:** Enables execution after task completion.
2. **Non-Blocking Execution:** Allows other operations to continue.
3. **Flexible Code Structure:** Functions can perform different actions through callbacks.
4. **Supports Event Handling:** Essential for interactive web applications.
5. **Simple Implementation:** Easy to use for small asynchronous tasks.

Disadvantages of Callback Functions

1. **Callback Hell:** Deep nesting reduces readability.
2. **Difficult Maintenance:** Complex callback structures are harder to manage.
3. **Complex Error Handling:** Managing errors can be challenging.
4. **Reduced Readability:** Large callback chains become difficult to understand.
5. **Limited Scalability:** Not suitable for highly complex asynchronous workflows.

Promises in JavaScript

A Promise is a JavaScript object that represents the eventual outcome of an asynchronous operation.

The outcome may be:

- Successful completion of the operation.
- Failure of the operation.

Since asynchronous tasks may require time to complete, a Promise serves as a placeholder until the final result becomes available.

Real-Life Analogy

Consider ordering a product online.

- The order confirmation acts as a Promise.
- The order is initially being processed.
- Eventually, the product is delivered successfully or the order is cancelled.

Similarly, a JavaScript Promise represents a future result that may either be fulfilled or rejected.

“A **Promise** is a JavaScript object that represents the eventual completion or failure of an asynchronous operation and its resulting value.”

Characteristics of Promises

Promises possess several important characteristics:

- Represent future values.
- Handle asynchronous operations efficiently.
- Improve code readability.
- Support method chaining.
- Facilitate centralized error management.
- Serve as the foundation for Async/Await.
- Eliminate excessive callback nesting.

Promise States

A Promise can exist in one of three states throughout its lifecycle.

1. Pending State

The Pending state is the initial state of every Promise.

Characteristics:

- The operation has started.
- No result is available yet.
- The Promise is waiting for completion.

Example

```
const promise =  
new Promise(function(resolve, reject) {  
  
});
```

At this stage, the Promise remains pending.

2. Fulfilled State

A Promise becomes fulfilled when the asynchronous operation completes successfully.

Characteristics:

- A result value becomes available.
- The Promise is resolved successfully.
- The success handler executes.

Example

```
resolve("Operation Successful");
```

3. Rejected State

A Promise becomes rejected when an error occurs during execution.

Characteristics:

- The operation fails.
- An error reason is provided.
- Error handling mechanisms execute.

Example

```
reject("Operation Failed");
```

Once a Promise enters the Fulfilled or Rejected state, its state becomes permanent and cannot be changed.

Creating a Promise

Promises are created using the Promise constructor.

Syntax

```
let promise =  
new Promise(function(resolve, reject) {  
  
    // asynchronous task  
  
});
```

Constructor Parameters

resolve()

The resolve() function is invoked when the operation completes successfully.

```
resolve(value);
```

reject()

The reject() function is invoked when the operation fails.

```
reject(error);
```

Example: Creating a Promise

```
const studentPromise =  
new Promise(function(resolve, reject) {  
  
    let success = true;  
  
    if(success) {  
  
        resolve(  
            "Student data loaded successfully"  
        );  
  
    }  
    else {  
  
        reject(  
            "Unable to load student data"  
        );  
  
    }  
  
});
```

Consuming a Promise

Creating a Promise alone is insufficient. The Promise result must be processed using Promise methods.

Promise Methods

JavaScript provides several methods for handling Promise outcomes.

Method	Purpose
then()	Handles successful execution
catch()	Handles errors
finally()	Executes regardless of outcome

The then() Method

Purpose

The then() method executes when a Promise is fulfilled successfully.

Syntax

```
promise.then(  
    function(result) {  
  
    }  
);
```

Example

```
Promise.resolve(  
  "Welcome Students"  
)  
  
.then(function(result) {  
  
  console.log(result);  
  
});
```

Output

Welcome Students

Explanation

- Promise resolves successfully.
- The resolved value is passed to then().
- The result is displayed.

The catch() Method

Purpose

The catch() method handles rejected Promises.

Syntax

```
promise.catch(  
  function(error) {  
  
  }  
);
```

Example

```
Promise.reject(  
  "Connection Error"  
)  
  
.catch(function(error) {  
  
  console.log(error);  
  
});
```

Output

Connection Error

Explanation

- Promise is rejected.
- Error information is passed to catch().
- Error message is displayed.

The finally() Method

Purpose

The finally() method executes regardless of whether the Promise succeeds or fails.

Syntax

```
promise.finally(  
  function() {  
  
  }  
);
```

Example

```
Promise.resolve("Success")  
  
  .finally(function() {  
  
    console.log(  
      "Operation Completed"  
    );  
  
  });
```

Output

Operation Completed

Promise Chaining

Promise chaining allows multiple asynchronous operations to execute sequentially.

The output of one Promise operation becomes the input of the next operation.

Example

```
Promise.resolve(5)  
  
  .then(function(number) {  
  
    return number * 2;  
  
  })  
  
  .then(function(number) {  
  
    return number + 10;  
  
  })  
  
  .then(function(result) {
```

```
console.log(result);
```

```
});
```

Output

20

Benefits of Chaining

- Reduces callback nesting.
- Improves readability.
- Simplifies sequential operations.
- Enhances maintainability.

Handling Asynchronous Operations Using Promises

Promises are particularly useful when operations take time to complete.

Example

```
const dataPromise =  
new Promise(function(resolve) {  
  
    setTimeout(function() {  
  
        resolve(  
            "Student Data Retrieved"  
        );  
  
    }, 3000);  
  
});  
  
dataPromise.then(function(data) {  
  
    console.log(data);  
  
});
```

Output After 3 Seconds

Student Data Retrieved

Error Handling with Promises

One of the major advantages of Promises is centralized error management.

Example

```
const promise =  
new Promise(function(resolve, reject) {  
  
    reject(  
        "Network Connection Failed"  
    );  
  
});  
  
promise  
  
.catch(function(error) {  
  
    console.log(error);  
  
});
```

Output

Network Connection Failed

Promise Utility Methods

Promise.resolve()

Creates an immediately fulfilled Promise.

Example

```
Promise.resolve("Success")  
  
.then(function(result) {  
  
    console.log(result);  
  
});
```

Promise.reject()

Creates an immediately rejected Promise.

Example

```
Promise.reject("Failure")  
  
.catch(function(error) {  
  
    console.log(error);  
  
});
```

Promise.all()

Executes multiple Promises concurrently and waits for all to complete.

Example

```
Promise.all([  
  
  Promise.resolve("A"),  
  Promise.resolve("B"),  
  Promise.resolve("C")  
  
])  
  
.then(function(result) {  
  
  console.log(result);  
  
});
```

Output

```
["A", "B", "C"]
```

Promise.race()

Returns the first Promise that settles.

Example

```
Promise.race([  
  
  new Promise(resolve =>  
    setTimeout(  
      () => resolve("First"),  
      1000  
    )  
  ),  
  
  new Promise(resolve =>  
    setTimeout(  
      () => resolve("Second"),  
      2000  
    )  
  )  
  
])  
  
.then(result =>  
  console.log(result)  
);
```

Output

First

Advantages of Promises

Advantage	Description
-----------	-------------

Improved Readability	Produces cleaner code
Better Error Handling	Centralized error management
Avoids Callback Hell	Reduces nested callbacks
Supports Chaining	Sequential asynchronous execution
Easier Maintenance	Simplifies debugging and updates
Foundation for Async/Await	Supports modern JavaScript development

Limitations of Promises

Limitation	Description
Learning Curve	Requires understanding asynchronous concepts
Additional Syntax	More complex than synchronous code
Long Promise Chains	Can become difficult to manage in very large applications

Applications of Promises

Promises are extensively used in modern web development for:

- AJAX requests
- API communication
- Database access
- Cloud services
- Authentication systems
- File processing
- Network communication
- Real-time applications
- Data retrieval operations

Async and Await in JavaScript

JavaScript applications often perform tasks that take time to complete, such as retrieving data from a server, reading files, accessing databases, or waiting for user input. These tasks are known as **asynchronous operations** because they do not produce results immediately.

Since JavaScript is a **single-threaded** programming language, executing time-consuming operations synchronously would block the execution of other code and make web applications unresponsive. To overcome this limitation, JavaScript provides asynchronous programming mechanisms.

Async/Await is a modern feature introduced in **ECMAScript 2017 (ES8)** that simplifies asynchronous programming by allowing developers to write asynchronous code in a clear and structured manner. It is built on top of **Promises** and provides a syntax that resembles traditional synchronous programming, making code easier to read, write, debug, and maintain.

The `async` keyword is used to declare an asynchronous function that automatically returns a Promise, while the `await` keyword is used within an `async` function to pause execution until a Promise is resolved or rejected. Together, these keywords provide a powerful and efficient way to handle asynchronous operations without relying on complex callback functions or lengthy Promise chains.

Async/Await has become the preferred approach for managing asynchronous tasks in modern JavaScript applications because it improves code readability, simplifies error handling through `try...catch` blocks, and helps developers build responsive and maintainable software systems.

Need for Async/Await

Before the introduction of Async/Await, JavaScript developers mainly used:

1. Callbacks
2. Promises

Although Promises improved code organization, complex asynchronous operations often resulted in lengthy `.then()` chains that were difficult to read and maintain.

Promise Chaining Example

```
fetchData()
  .then(data => processData(data))
  .then(result => displayData(result))
  .catch(error => console.error(error));
```

As the application grows, managing multiple `.then()` statements becomes challenging.

Async/Await Approach

```
async function getData() {
  try {
    const data = await fetchData();
    const result = processData(data);
    displayData(result);
  }
  catch(error) {
    console.error(error);
  }
}
```

This approach resembles traditional sequential programming and is easier to understand.

Async Function

An **async function** is a function declared using the `async` keyword.

The primary purpose of an async function is to simplify asynchronous programming by enabling the use of the `await` keyword within the function body.

Important Characteristics

- Always returns a Promise.
- Makes asynchronous code easier to read.
- Can contain one or more `await` expressions.
- Does not block the execution of other JavaScript code.
- Automatically wraps non-Promise return values into resolved Promises.

Syntax

```
async function functionName() {  
  
}
```

Arrow Function Syntax

```
const functionName = async () => {  
  
};
```

How Async Functions Return Values

Example

```
async function greet() {  
    return "Hello Students";  
}  
  
greet().then(message => {  
    console.log(message);  
});
```

Output

Hello Students

Explanation

Although the function returns a simple string, JavaScript automatically converts it into:

```
Promise.resolve("Hello Students")
```

Therefore, every async function returns a Promise regardless of the return value

Return Types of Async Functions

Return Statement	Actual Returned Value
return 10	Promise resolving to 10
return "Hello"	Promise resolving to "Hello"
return true	Promise resolving to true
return Promise.resolve(100)	Promise resolving to 100

Await Keyword

The await keyword is used inside an async function to pause its execution until a Promise is settled (resolved or rejected).

Instead of using .then() to obtain Promise results, await allows the result to be assigned directly to a variable.

Syntax

```
let result = await promise;
```

Characteristics of Await

- Can only be used inside an async function.
- Waits for a Promise to complete.
- Returns the resolved value of the Promise.
- Makes asynchronous code appear synchronous.
- Improves readability and maintainability.

Example

```
const getMessage = async () => {

  let result = await Promise.resolve(
    "Welcome to JavaScript"
  );

  console.log(result);
};
```

```
getMessage();
```

Output

Welcome to JavaScript

Execution Flow of Await

Consider the following example:

```
const getData = async () => {

  let result =
    await Promise.resolve("Hello");

  console.log(result);
};
```

```
console.log("Start");
```

```
getData();
```

```
console.log("End");
```

Output

Start

End

Hello

Explanation

1. "Start" is displayed.
2. getData() is called.
3. The async function encounters await.
4. JavaScript continues executing remaining statements.
5. "End" is displayed.
6. After the Promise resolves, "Hello" is displayed.

This demonstrates that await pauses only the async function, not the entire program.

Relationship Between Async and Await

The async and await keywords work together.

async	await
Creates an asynchronous function	Waits for a Promise
Enables asynchronous programming	Retrieves Promise results
Returns a Promise	Can only be used inside async functions
Required for using await	Depends on async functions

Working with Promises Using Async/Await

Traditional Promise Example

```
function getData() {  
  
    return Promise.resolve(  
        "JavaScript Promise"  
    );  
  
}  
  
getData()  
    .then(data => {  
        console.log(data);  
    });
```

Async/Await Version

```
function getData() {  
  
    return Promise.resolve(  
        "JavaScript Promise"  
    );  
  
}  
  
async function displayData() {  
  
    let result = await getData();  
  
    console.log(result);  
  
}
```

```
displayData();
```

The second version is easier to read and understand.

Real-World Example: Fetching Data from an API

```
async function fetchData() {  
  try {  
    const response = await fetch(  
      "https://jsonplaceholder.typicode.com/posts/1"  
    );  
    const data = await response.json();  
    console.log(data);  
  }  
  catch(error) {  
    console.error(error);  
  }  
}  
  
fetchData();
```

Step-by-Step Explanation

Step 1

```
const response = await fetch(url);
```

An HTTP request is sent to the server.

Step 2

```
const data = await response.json();
```

The JSON response is converted into a JavaScript object.

Step 3

```
console.log(data);
```

The retrieved data is displayed.

Step 4

```
catch(error)
```

Any error occurring during the request is handled.

Error Handling in Async/Await

Errors are inevitable in software applications. Network failures, invalid URLs, and server issues may cause asynchronous operations to fail.

Async/Await uses the familiar try...catch structure for error handling.

Syntax

```
async function functionName() {  
  
  try {  
  
    let result =  
      await somePromise();  
  
  }  
  catch(error) {  
  
    console.log(error);  
  
  }  
  
}
```

Example

```
async function example() {  
  
  try {  
  
    let result =  
      await Promise.reject(  
        "Operation Failed"  
      );  
  
    console.log(result);  
  
  }  
  catch(error) {  
  
    console.log(error);  
  
  }  
  
}
```

example();

Output

Operation Failed

Multiple Await Statements

An async function may contain multiple await expressions.

```
async function processData() {  
  
  let value1 =
```



```
    await Promise.resolve(10);

    let value2 =
      await Promise.resolve(20);

    console.log(value1 + value2);
  }

  processData();
```

Output

30

Async/Await with Promise.all()

When multiple asynchronous tasks are independent, they can be executed simultaneously using Promise.all().

Sequential Execution

```
let data1 = await getData1();
let data2 = await getData2();
```

The second task starts only after the first task finishes.

Parallel Execution

```
let [data1, data2] =
  await Promise.all([
    getData1(),
    getData2()
  ]);
```

Both operations run concurrently, improving performance.

Advantages of Async/Await

Advantage	Description
Improved Readability	Code resembles synchronous programming
Better Maintenance	Easier to modify and debug
Reduced Complexity	Eliminates callback nesting
Efficient Error Handling	Uses try...catch
Promise-Based	Built upon modern JavaScript Promises
Cleaner Syntax	Less code compared to Promise chains

Limitations of Async/Await

Limitation	Description
Requires Promises	Works only with Promise-based operations
Sequential Execution	Multiple awaits may reduce performance if used unnecessarily
Cannot Be Used Everywhere	Must be inside an async function
Older Browser Support	Requires modern JavaScript environments

XMLHttpRequest (XHR)

XMLHttpRequest (XHR) is a built-in JavaScript object that enables web applications to communicate with a web server asynchronously. It allows data to be sent to and received from a server without reloading the entire webpage. This capability makes web applications more interactive, responsive, and efficient.

XMLHttpRequest is the core technology behind **AJAX (Asynchronous JavaScript and XML)** and has been widely used for dynamic content loading in web applications. Although modern JavaScript applications often use the Fetch API, XMLHttpRequest remains important for understanding asynchronous communication and maintaining legacy systems.

XMLHttpRequest (XHR) is a browser-provided JavaScript object used to send HTTP requests and receive HTTP responses asynchronously from a web server.

It enables developers to:

- Retrieve data from a server.
- Send user data to a server.
- Update webpage content dynamically.
- Communicate with web services and APIs.
- Perform background data processing.

Features of XMLHttpRequest

Feature	Description
Asynchronous Communication	Communicates with servers in the background
Dynamic Content Updates	Updates specific webpage sections
HTTP Request Support	Supports GET, POST, PUT, DELETE methods
Multiple Data Formats	Handles XML, JSON, HTML, and Text
Event-Based Processing	Supports various request events
Browser Integration	Built into modern web browsers

Creating an XMLHttpRequest Object

Before communicating with a server, an XMLHttpRequest object must be created.

Syntax

```
let xhr = new XMLHttpRequest();
```

Example

```
const xhr = new XMLHttpRequest();
```

This statement creates a new XMLHttpRequest object that can be used for server communication.

Methods of XMLHttpRequest

Methods are functions that perform specific operations on the XMLHttpRequest object.

Common Methods

Method	Purpose
open()	Initializes a request
send()	Sends the request to the server
abort()	Cancels the current request
setRequestHeader()	Sets HTTP request headers
getResponseHeader()	Retrieves a specific response header
getAllResponseHeaders()	Retrieves all response headers

open() Method

The `open()` method configures the request before it is sent.

Syntax

```
xhr.open(method, url, async);
```

Parameters

Parameter	Description
method	HTTP request method
url	Server resource location
async	Indicates asynchronous execution

Example

```
xhr.open(  
    "GET",  
    "students.json",  
    true  
);
```

send() Method

The `send()` method transmits the configured request to the server.

Syntax

```
xhr.send();
```

Example

```
xhr.send();
```

Properties of XMLHttpRequest

Properties provide information about the request and response.

Important Properties

Property	Description
readyState	Current state of the request
status	HTTP status code
statusText	Status description
responseText	Response as text
responseXML	Response as XML
response	Complete response data

readyState Property

The `readyState` property indicates the current stage of the request.

Value	State	Description
0	UNSENT	Request not initialized
1	OPENED	Request initialized
2	HEADERS_RECEIVED	Server received request
3	LOADING	Response is being processed
4	DONE	Request completed

The value **4** indicates that the response has been completely received.

HTTP Status Codes

The `status` property indicates whether the request was successful.

Status Code	Meaning
200	OK (Success)
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

A successful request generally returns status code **200**.

Handling Responses

`onreadystatechange` Event

The `onreadystatechange` event is triggered whenever the `readyState` value changes.

Syntax

```
xhr.onreadystatechange = function() {
```

```
};
```

Example

```
xhr.onreadystatechange = function() {
```

```
    if (
        xhr.readyState === 4 &&
        xhr.status === 200
    ) {
```

```
        console.log(
            xhr.responseText
        );
```

```
    }
```

```
};
```

Explanation

- Checks whether the request is complete.

- Verifies successful execution.
- Displays the server response.

Complete Example: Loading Data from a Text File

data.txt

Welcome to XMLHttpRequest Programming.

HTML and JavaScript

<!DOCTYPE html>

<html>

<head>

<title>XHR Example</title>

</head>

<body>

<h2>XMLHttpRequest Example</h2>

<button onclick="loadData()">

Load Data

</button>

<p id="result"></p>

<script>

function loadData() {

const xhr =

new XMLHttpRequest();

xhr.onreadystatechange =

function() {

if (

xhr.readyState === 4 &&

xhr.status === 200

) {

document.getElementById(

"result"

).innerHTML =

xhr.responseText;

}

};

xhr.open(

"GET",

"data.txt",

true

```
);  
  
xhr.send();  
  
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Loading JSON Data Using XMLHttpRequest

students.json

```
[  
  {  
    "id": 1,  
    "name": "Ram Sharma"  
  },  
  {  
    "id": 2,  
    "name": "Sita Thapa"  
  }  
]
```

JavaScript Code

```
const xhr =  
  new XMLHttpRequest();  
  
xhr.onreadystatechange =  
function() {  
  
  if(  
    xhr.readyState === 4 &&  
    xhr.status === 200  
  ) {  
  
    const students =  
      JSON.parse(  
        xhr.responseText  
      );  
  
    console.log(students);  
  
  }  
  
};  
  
xhr.open(  
  "GET",  
  "students.json",  
  true  
);
```

```
xhr.send();
```

Advantages of XMLHttpRequest

Advantage	Description
Faster Communication	No page reload required
Improved User Experience	Dynamic content updates
Reduced Network Traffic	Transfers only necessary data
Background Processing	Performs asynchronous operations
Broad Browser Support	Supported by all major browsers

Limitations of XMLHttpRequest

Limitation	Description
Verbose Syntax	Requires more code than Fetch API
Callback-Based	Can lead to callback nesting
Complex Error Handling	More difficult than async/await
Legacy Technology	Replaced by modern Fetch API

Applications of XMLHttpRequest

XMLHttpRequest is commonly used in:

- AJAX applications
- Live search systems
- Form validation and submission
- Chat applications
- Social media platforms
- Real-time dashboards
- Online banking systems
- Dynamic content management systems